

LAB ACTION PLAN FOR WEEK 4

Objective:

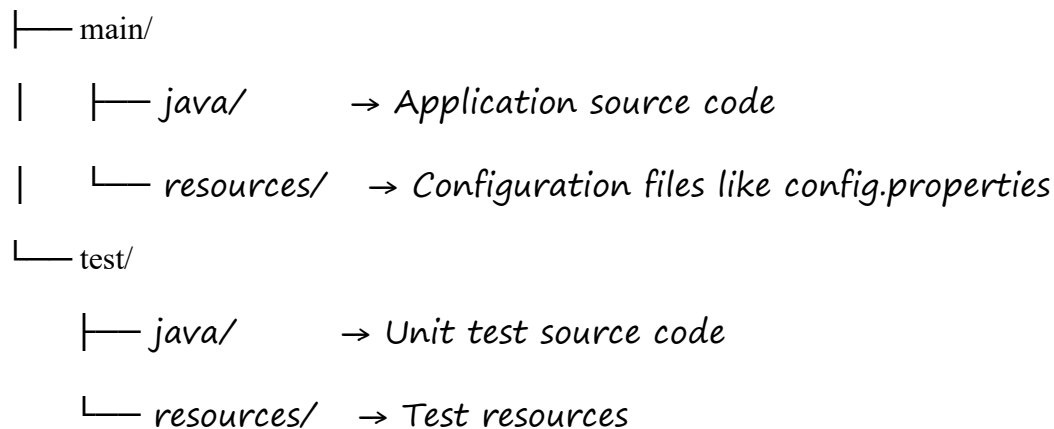
To enable students to:

- Understand the structure and lifecycle of a Maven project.
- Build and package Java and Web applications using Maven.
- Add dependencies using **pom.xml**, compile and test using plugins.
- Resolve errors and conflicts arising from dependency mismatches.
- Work with parent and multi-module Maven projects.
- Generate executable JARs and deployable WARs using Maven.

Students must **document observations**, include **screenshots of executions**, and answer **scenario-based questions** after completing the tasks.

1. Understanding Maven Project Structure

Maven standardizes the project structure for both Java and web-based applications. `src/`



The compiled files and reports are generated in the `target/` directory after a successful build.

2. Steps to Perform Maven Build and Testing

---mvn clean install

- **clean:** Deletes the previous build (`target/` folder)
- **install:** Builds, tests, and installs the package to local `.m2` repository

After execution:

- `target/` folder contains compiled `.class` files and the final `.jar` or `.war`

- Artifact is stored in:
`~/.m2/repository/groupId/artifactId/version/`

Add a Dependency (e.g., Gson):

In `pom.xml`:

```
<dependency>  
  <groupId>com.google.code.gson</groupId>  
  <artifactId>gson</artifactId>  
  <version>2.10</version>  
</dependency>
```

After running:

```
mvn clean install
```

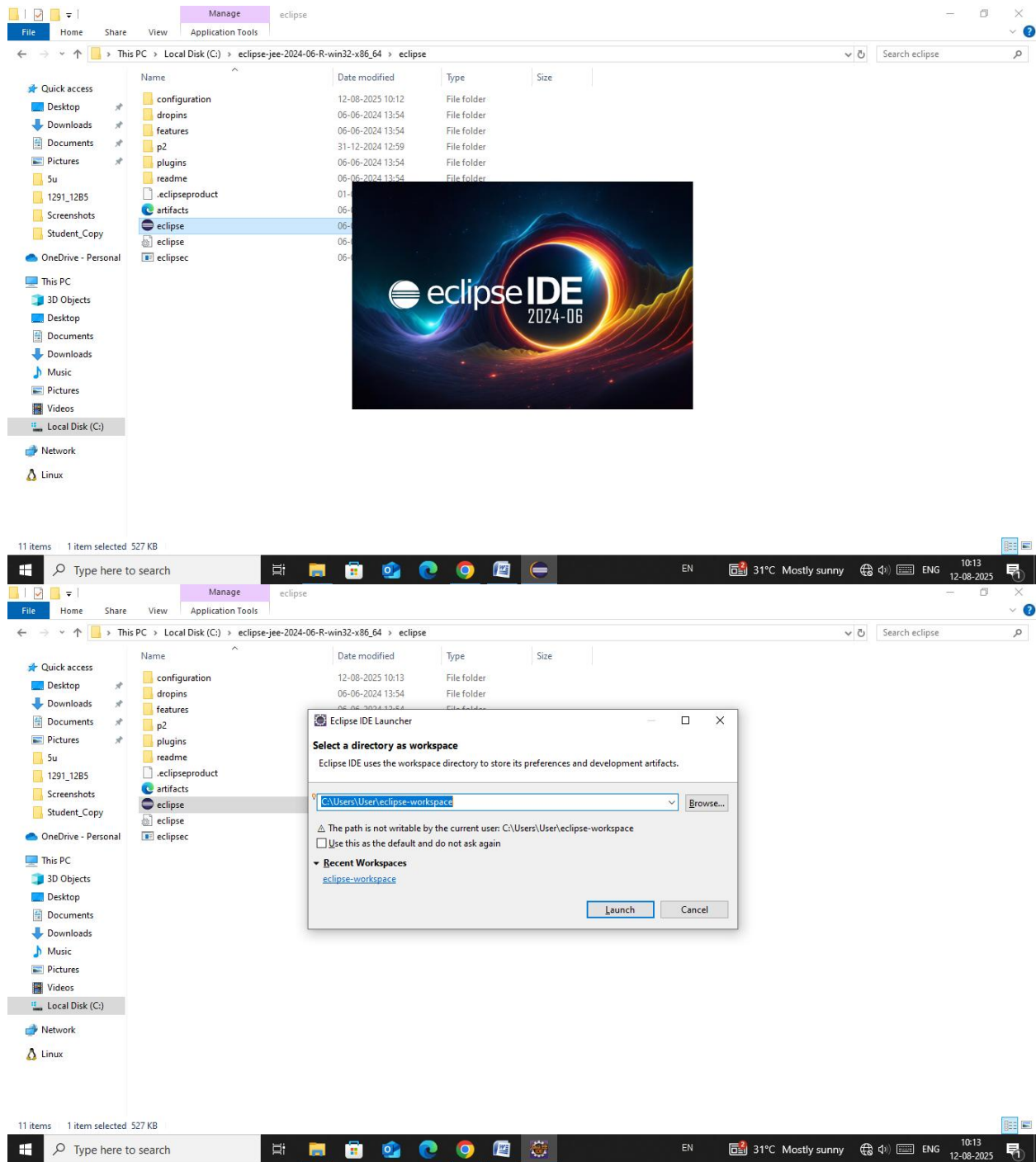
Check:

- Gson JAR is downloaded to `.m2/repository/com/google/code/gson/gson/`
- If version is wrong or not found → **BUILD FAILURE** with dependency resolution error.

Creation of Maven Java Project

Step 1. Open Eclipse IDE

└─ 1.1. Launch Eclipse workspace



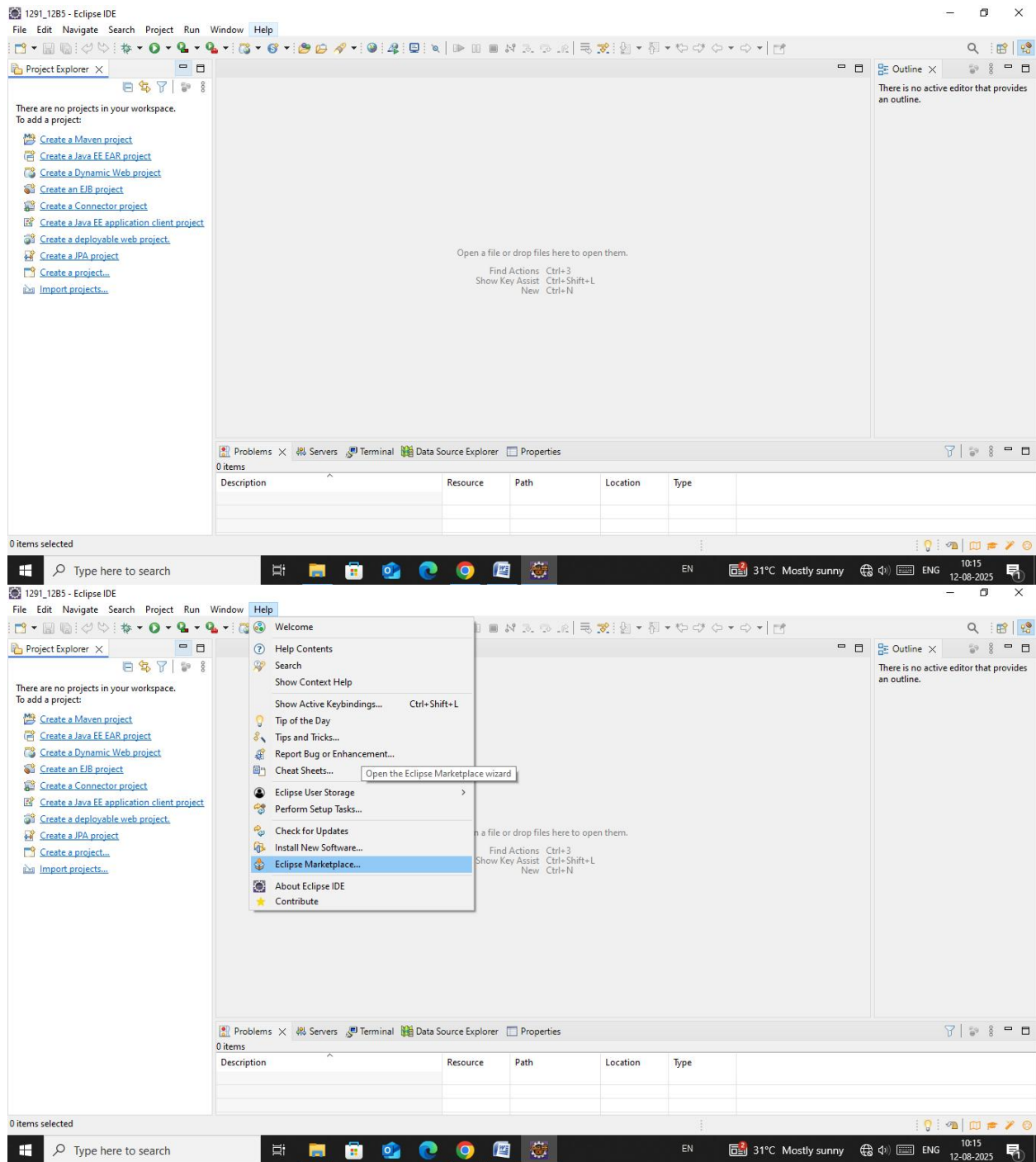
Step 2. Install Maven Plugin (if not installed)

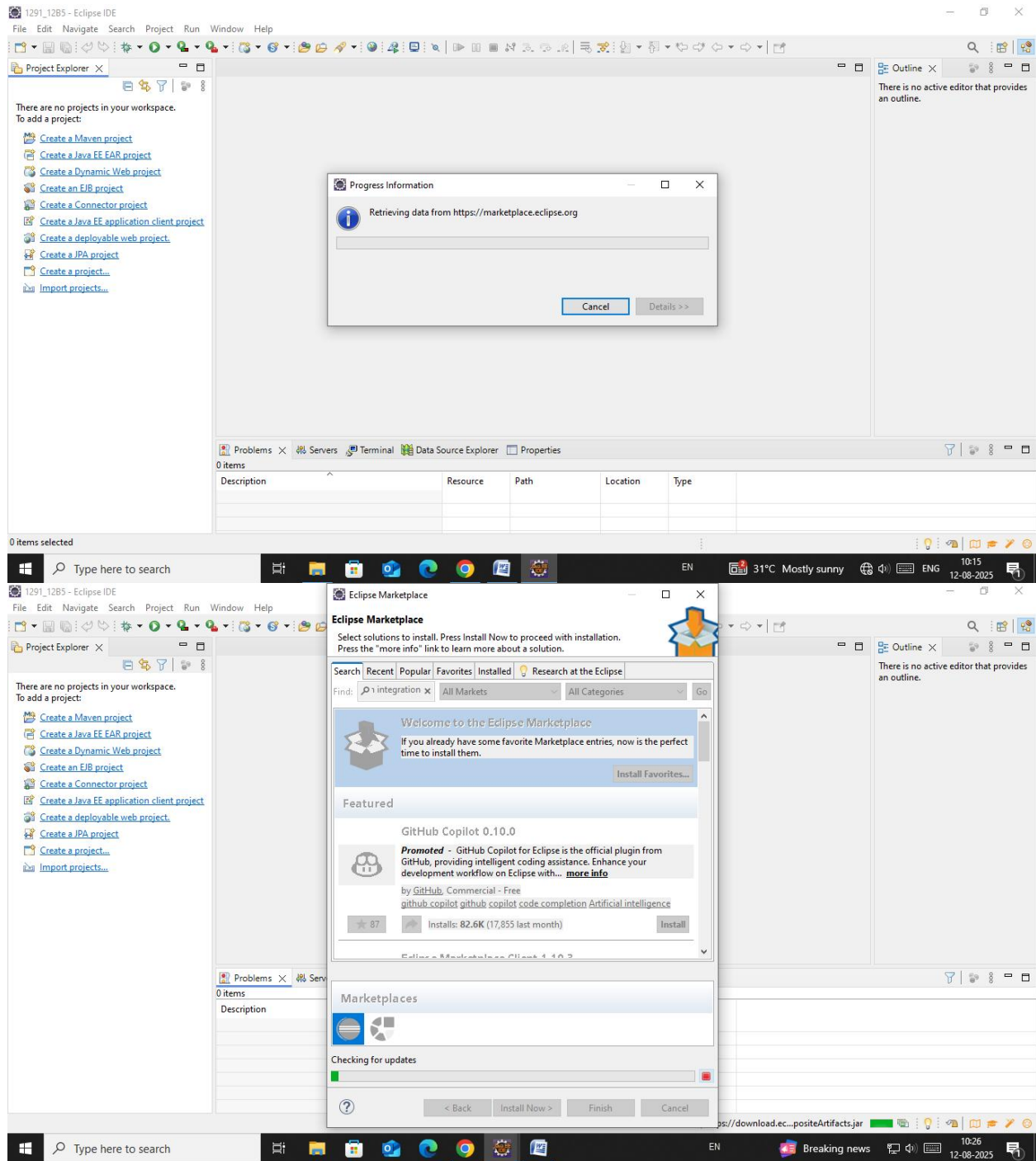
└─ 2.1. Go to "Help" in the top menu

└─ 2.1.1. Click "Eclipse Marketplace"

└─ 2.1.2. Search for "Maven Integration for Eclipse"

2.1.3. Install the plugin if not already installed





Step 3. Create a New Maven Project

└─ 3.1. File -> New -> Project...

└─ 3.1.1. Expand "Maven"

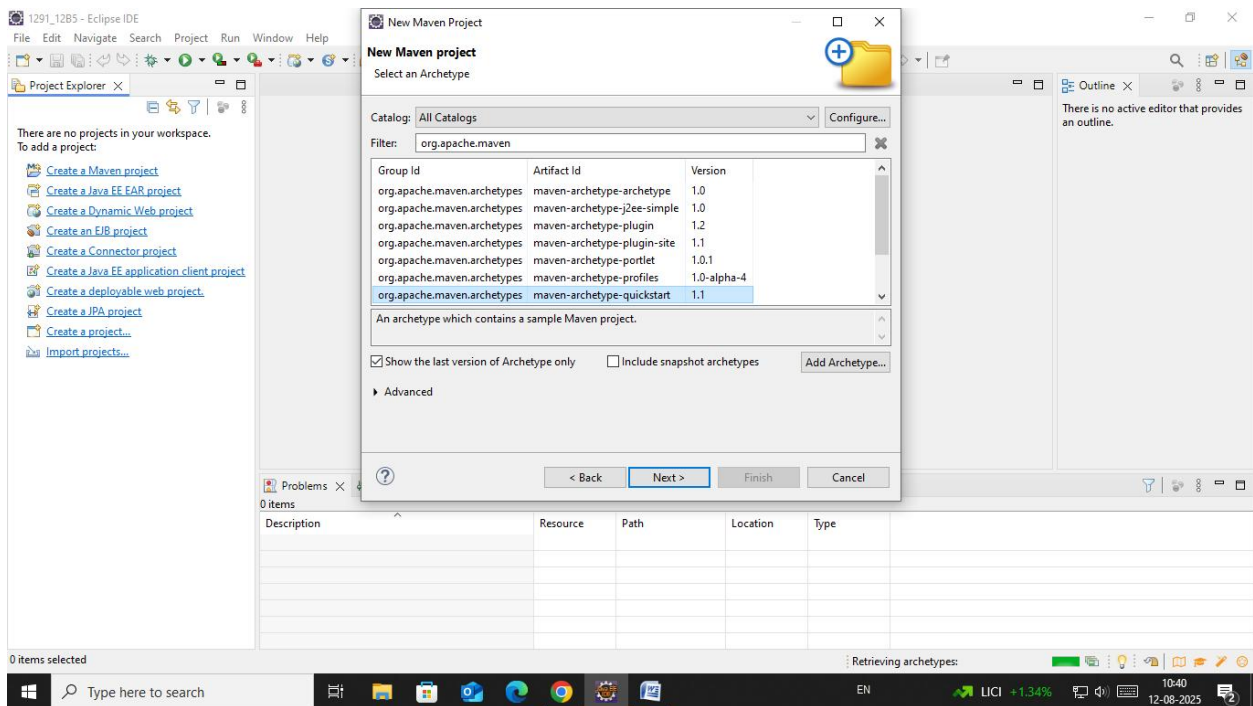
└─ 3.1.2. Select "Maven Project" and click "Next"

Step 4. Set Project Configuration

- └─ 4.1. Select workspace location (default or custom)
- └─ 4.2. Click "Next"

Step 5. Choose Maven Archetype

- └─ 5.1. Select an archetype(e.g "org.apache.maven.archetypes -> maven-archetype-quickstart 1.4 ")
- └─ 5.2. Click "Next"



Step 6. Define Project Metadata

- └─ 6.1. Group ID: (e.g., com.example)
- └─ 6.2. Artifact ID: (e.g., my-maven-project)
- └─ 6.3. Version: (default is usually fine)
- └─ 6.4. Click "Finish"

In Console, artifacts are grouped. When prompted with Y/N, type 'Y'.

1291_1285 - Eclipse IDE

File Edit Navigate Search Project Run Window Help

Project Explorer

There are no projects in your workspace. To add a project:

- Create a Maven project
- Create a Java EE EAR project
- Create a Dynamic Web project
- Create an EJB project
- Create a Connector project
- Create a Java EE application client project
- Create a deployable web project
- Create a JPA project
- Create a project...
- Import projects...

New Maven Project

Specify Archetype parameters

Group Id: com.lab

Artifact Id: mySElab

Version: 0.0.1-SNAPSHOT

Package: com.lab.mySElab

☒ run archetype generation interactively

Properties available from archetype:

Name	Value

Advanced

< Back Next > Finish Cancel

0 items

Retrieving archetypes:

1291_1285 - Eclipse IDE

File Edit Navigate Search Project Run Window Help

Project Explorer

There are no projects in your workspace. To add a project:

- Create a Maven project
- Create a Java EE EAR project
- Create a Dynamic Web project
- Create an EJB project
- Create a Connector project
- Create a Java EE application client project
- Create a deployable web project
- Create a JPA project
- Create a project...
- Import projects...

Open a file or drop files here to open them.

Find Actions Ctrl+3
Show Key Assist Ctrl+Shift+L
New Ctrl+N

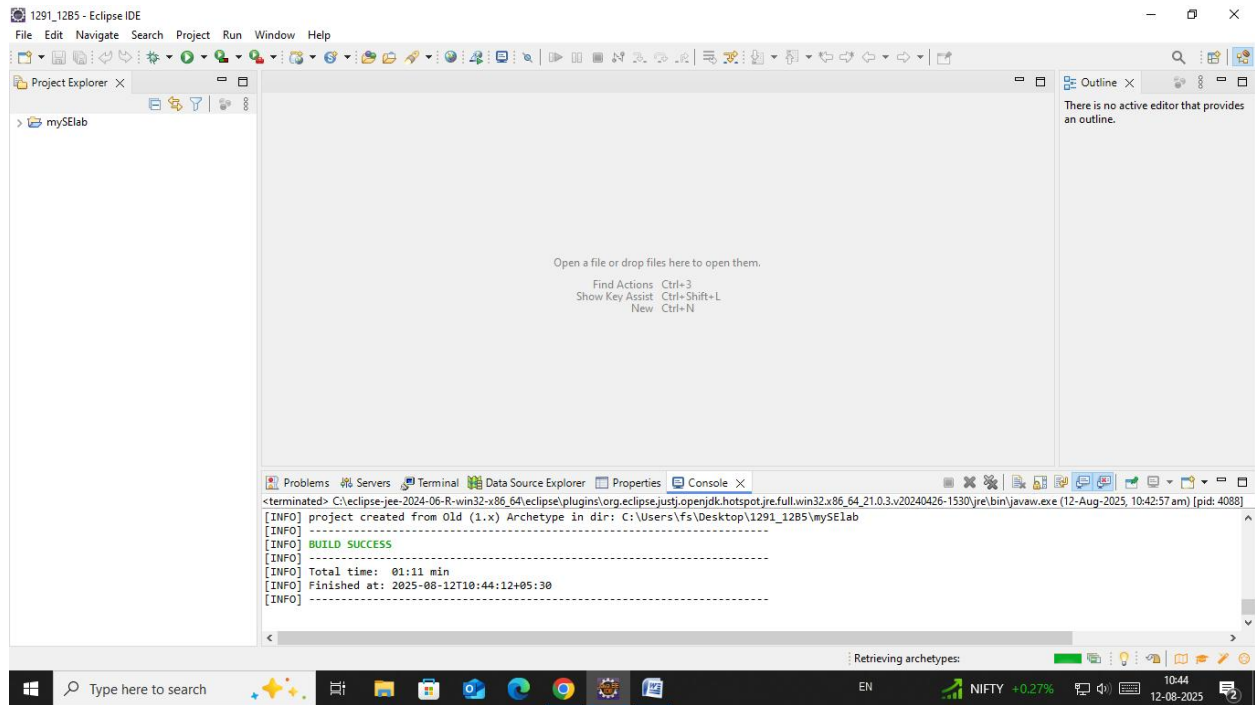
Problems Servers Terminal Data Source Explorer Properties Console

C:\eclipse-jee-2024-06-R-win32-x86_64\eclipse\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.21.0.3.v20240426-1530\jre\bin\javaw.exe (12-Aug-2025, 10:42:57 am) [pid: 4088]

[INFO] Using property: version = 0.0.1-SNAPSHOT
[INFO] Using property: package = com.lab.mySElab
Confirm properties configuration:
groupId: com.lab
artifactId: mySElab
version: 0.0.1-SNAPSHOT
package: com.lab.mySElab
Y: y

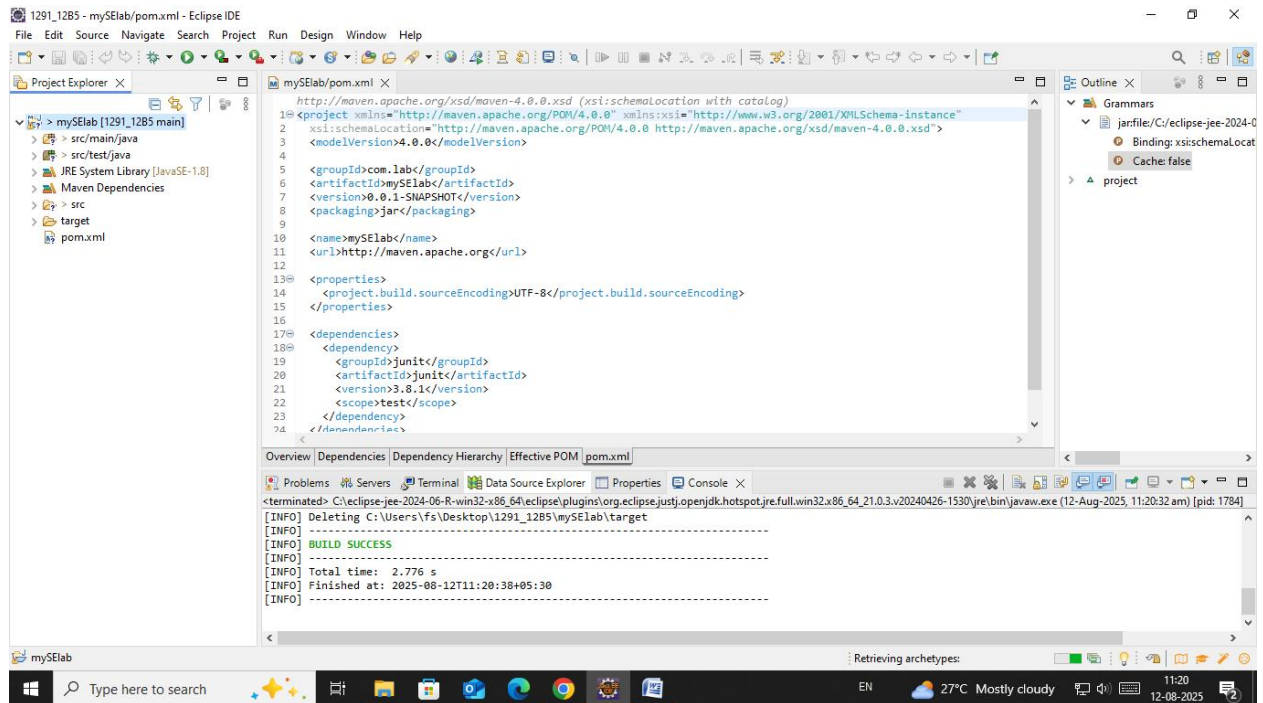
Retrieving archetypes:

10:44
12-08-2025



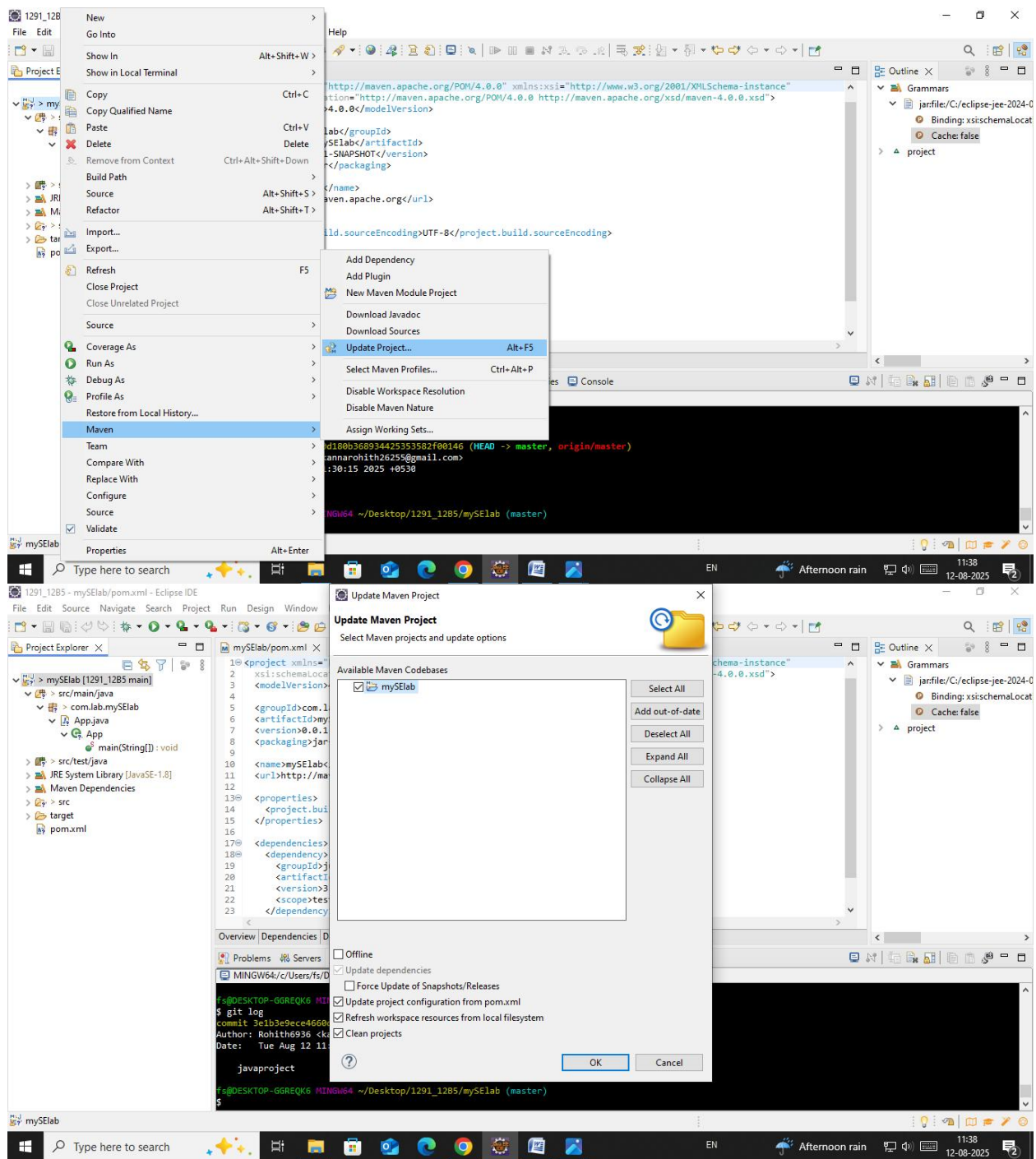
Step 7. Maven Project Created

- └─ 7.1. Project structure is generated with a standard Maven layout
- └─ 7.2. Includes:
 - └─ src/main/java (for Java source code)
 - └─ src/test/java (for test code)
 - └─ pom.xml (Maven configuration file)



Step 8. Update Project Settings (if needed)

- └─ 8.1. Right-click on the project -> Maven -> Update Project...
- └─ 8.2. Ensure dependencies are up to date



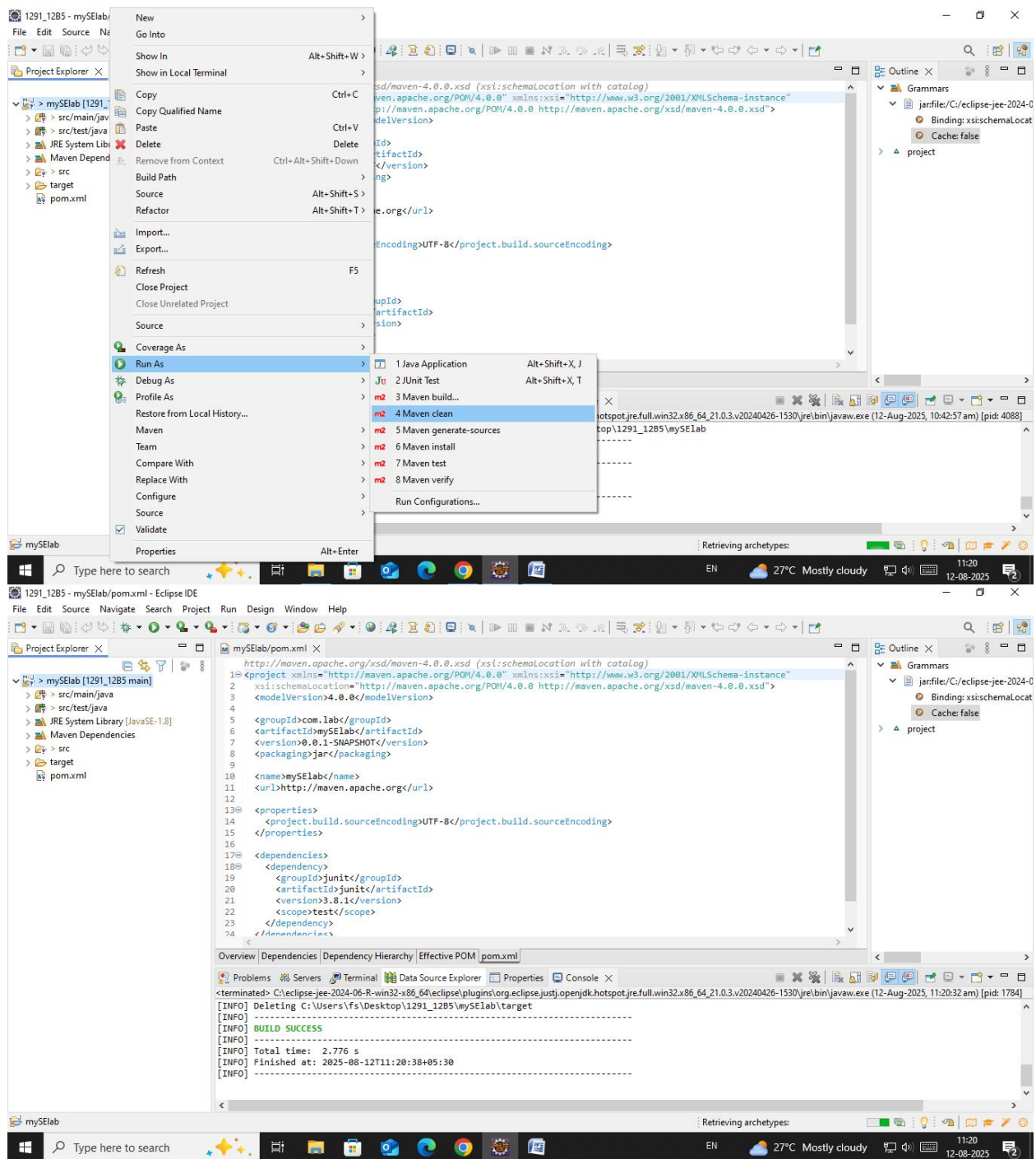
Step 9. Build and Run Maven Project

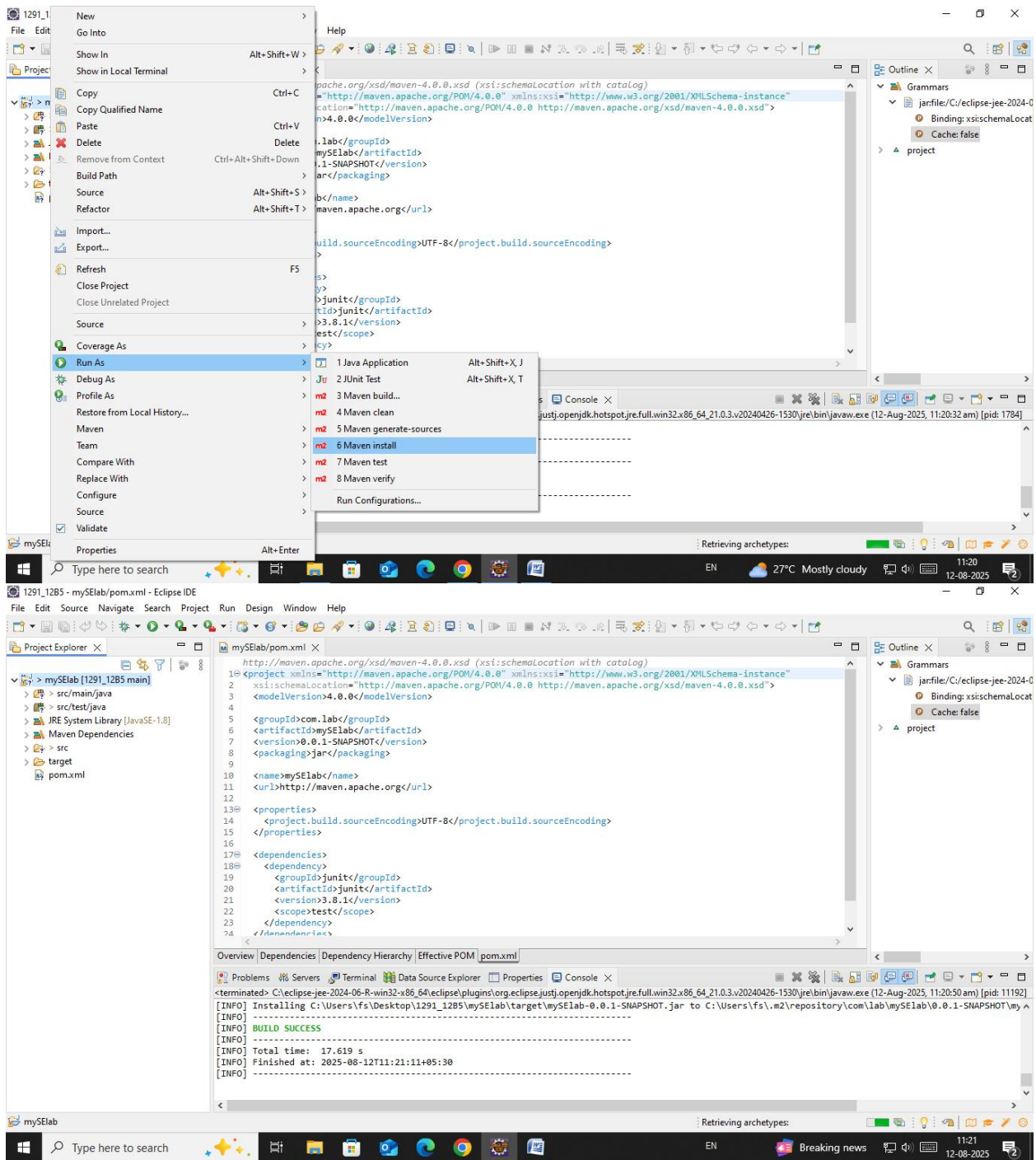
└─ 9.1. Right-click on App.java -> Run As -> Maven Clean

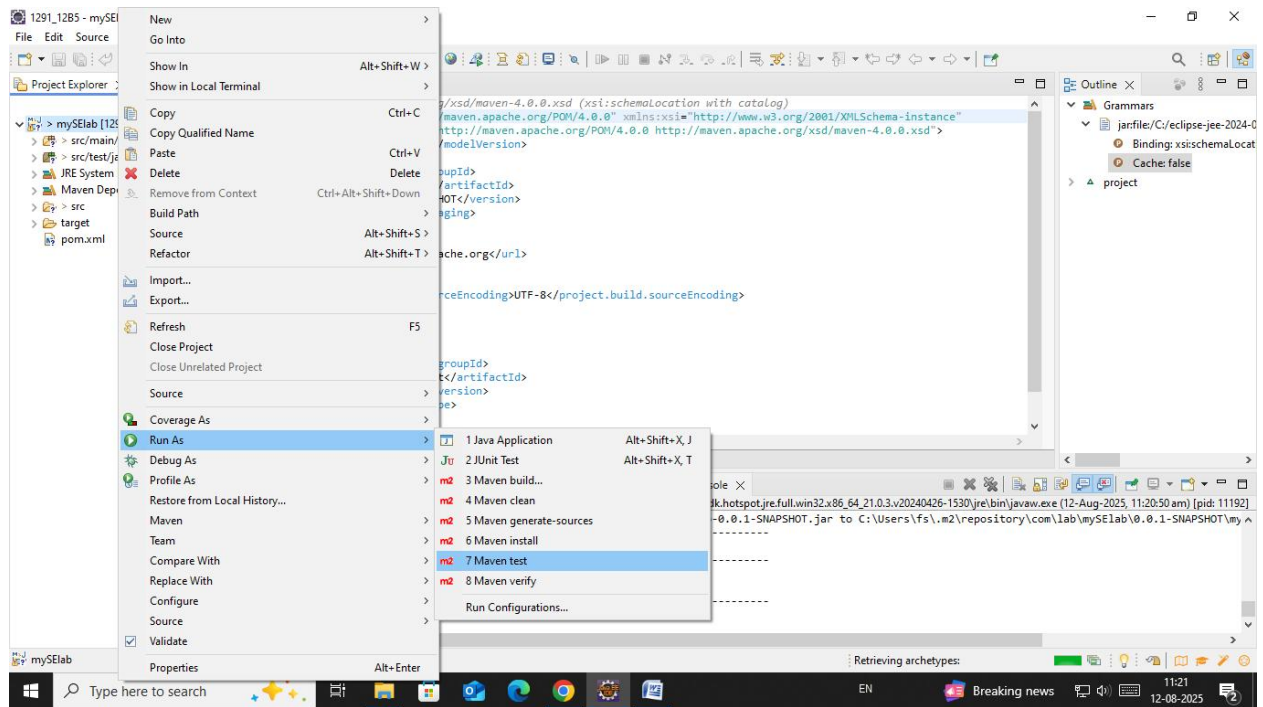
└─ 9.1.1. Right-click on App.java -> Run As -> Maven Install

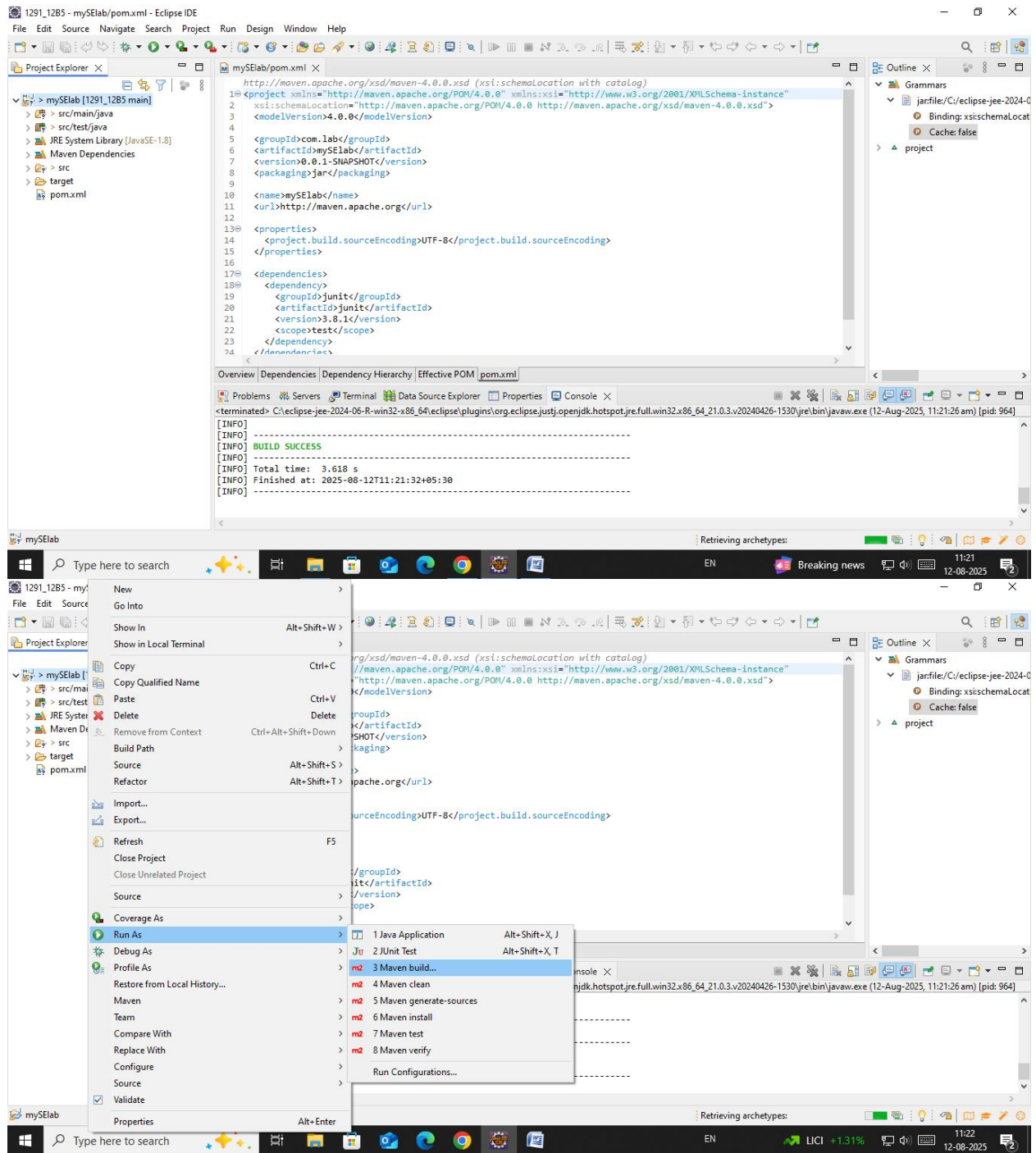
└─ 9.1.2. Right-click on App.java -> Run As -> Maven Test

9.1.3. Right-click on App.java -> Run As -> Maven Build





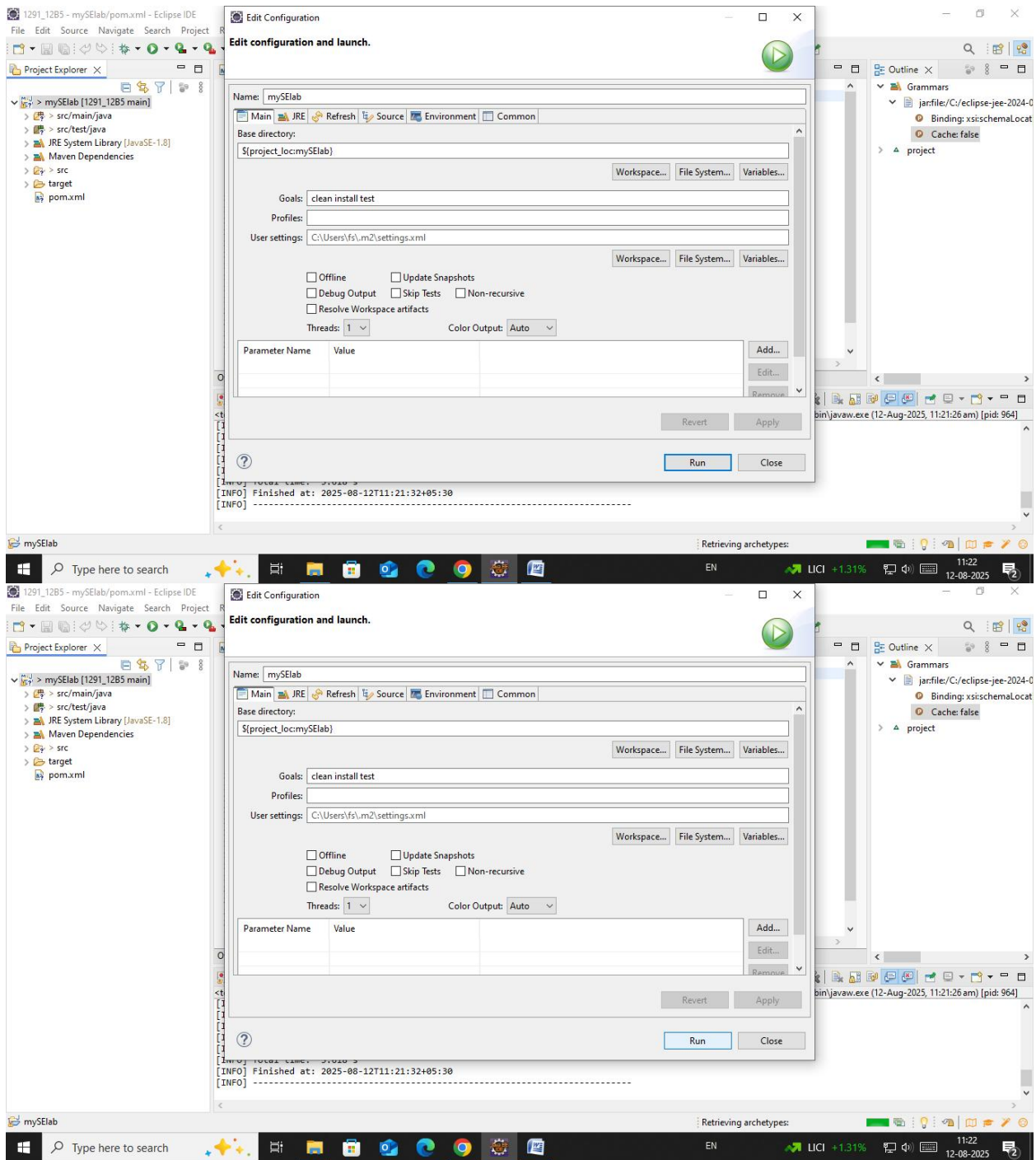




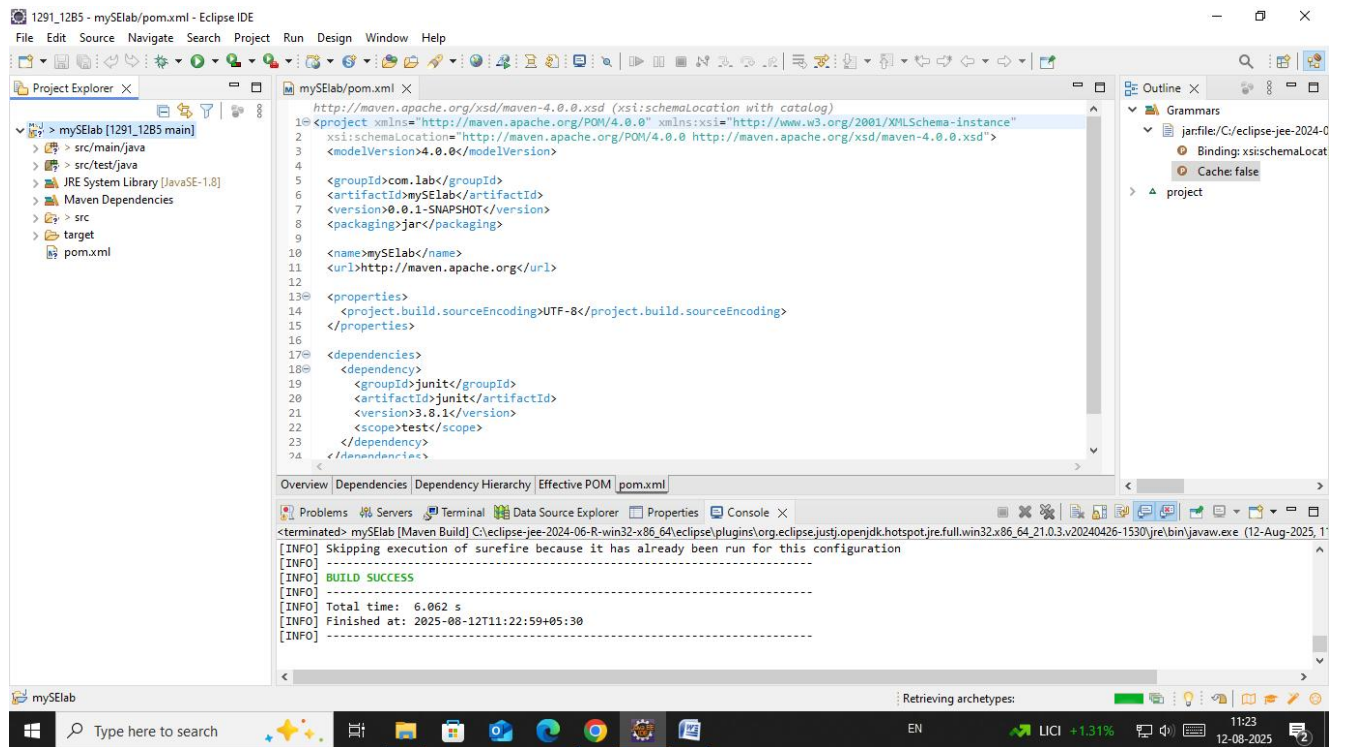
Step 10. In the Maven Build dialog:

└ Enter Goals: clean install test

└ Click on Apply -> Click on Run



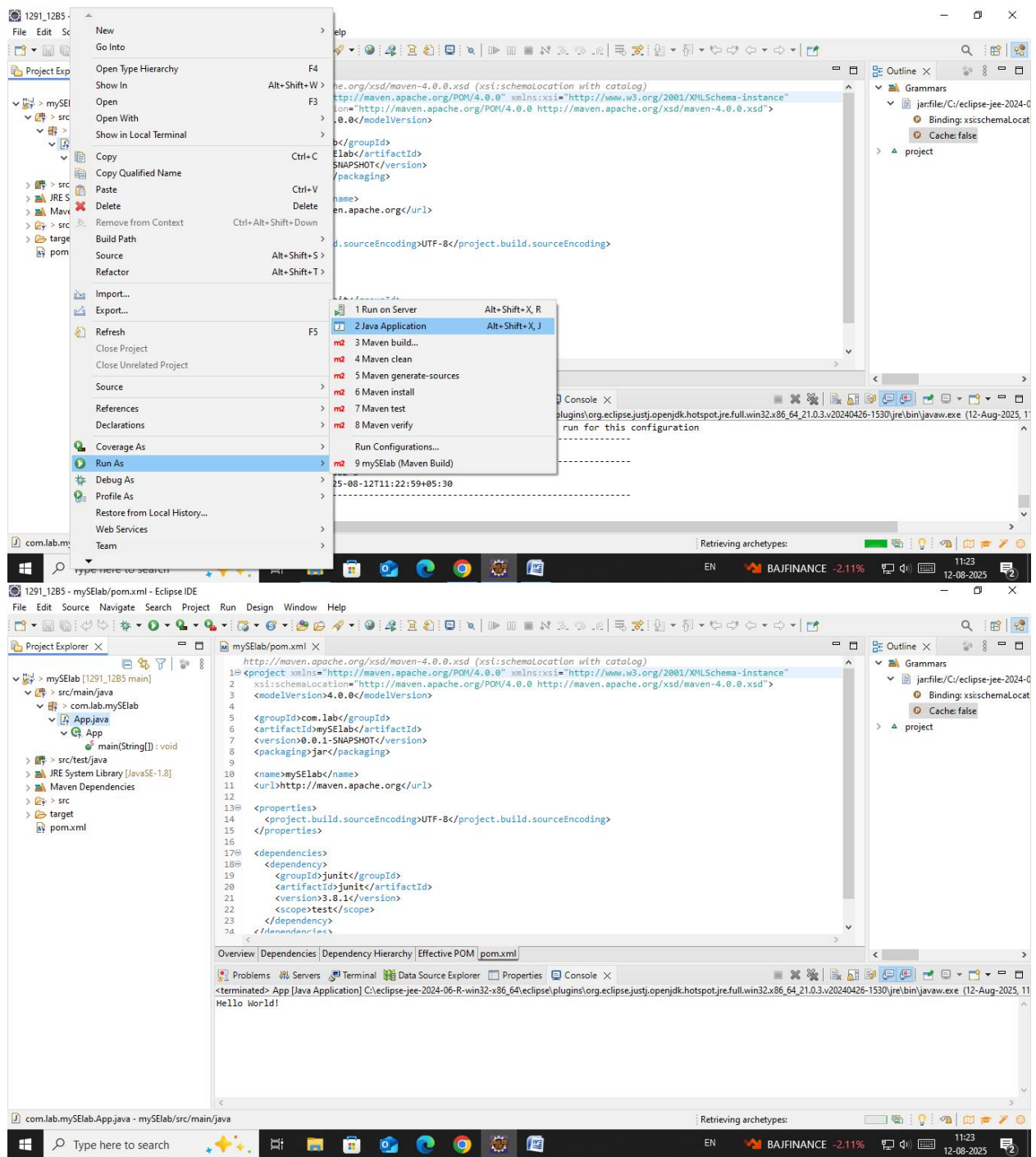
Step 11. Check console for BUILD SUCCESS message.



Step 12. Run the application:

— Right-click on App.java -> Run As -> Java Application

— Output: "Hello World" displayed.



GITHUB:

Step: create a repository

tesse

Log in to the site

GitHub

github.com

Dashboard

Q Type [Z] to search

⚠

GitHub users are [now required](#) to enable two-factor authentication as an additional security measure. Your activity on GitHub includes you in this requirement. You will need to enable two-factor authentication on your account before September 20, 2025, or be restricted from account actions.

Rohith6936

Top repositories

Find a repository...

Rohith6936/project

SELAB1291B5/project

Rohith6936/SE_LAB_1291

Rohith6936/face-recognition

Rohith6936/helloworld

Rohith6936/project1

Rohith6936/student-portal

Show more

Home

Ask Copilot

Create a profile README for me

Create an issue for a bug

My open pull requests

Feed

Filter

Trending repositories · See more

mendableai/open-lovable

Clone and recreate any website as a modern React app in seconds

TypeScript · 5k

nomic-ai/gpt4all

GPT4All: Run Local LLMs on Any Device. Open-source and available for commercial use.

New issue

New repository

Import repository

New codespace

New gist

New organization

New project

Upcoming changes to GitHub Events API payloads

Defense of third-party claims added for volume licensing...

GPT-5 is now generally available in GitHub Models

View changelog

https://github.com/new

SE_LAB_1291_12b5

SE_LAB_1291_12b5 is available.

Great repository names are short and memorable. How about [jubilant-octo-succotash](#)?

Description

0 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

Maven

Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

Type here to search

EN

NIFTY -0.12%

11:24

12-08-2025

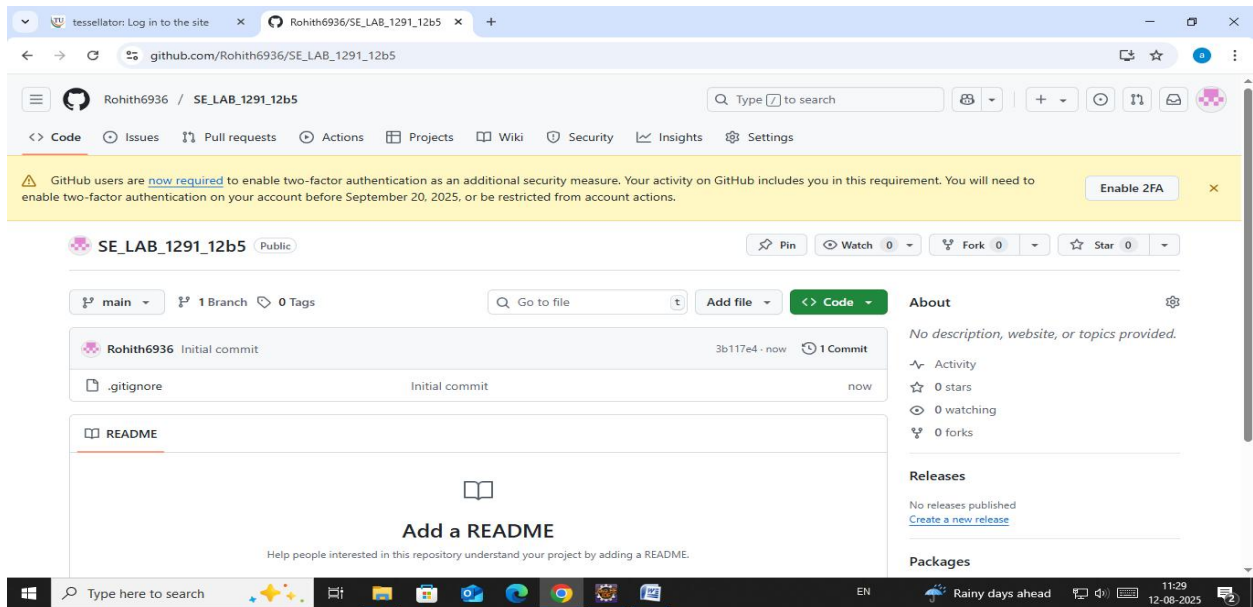
Type here to search

EN

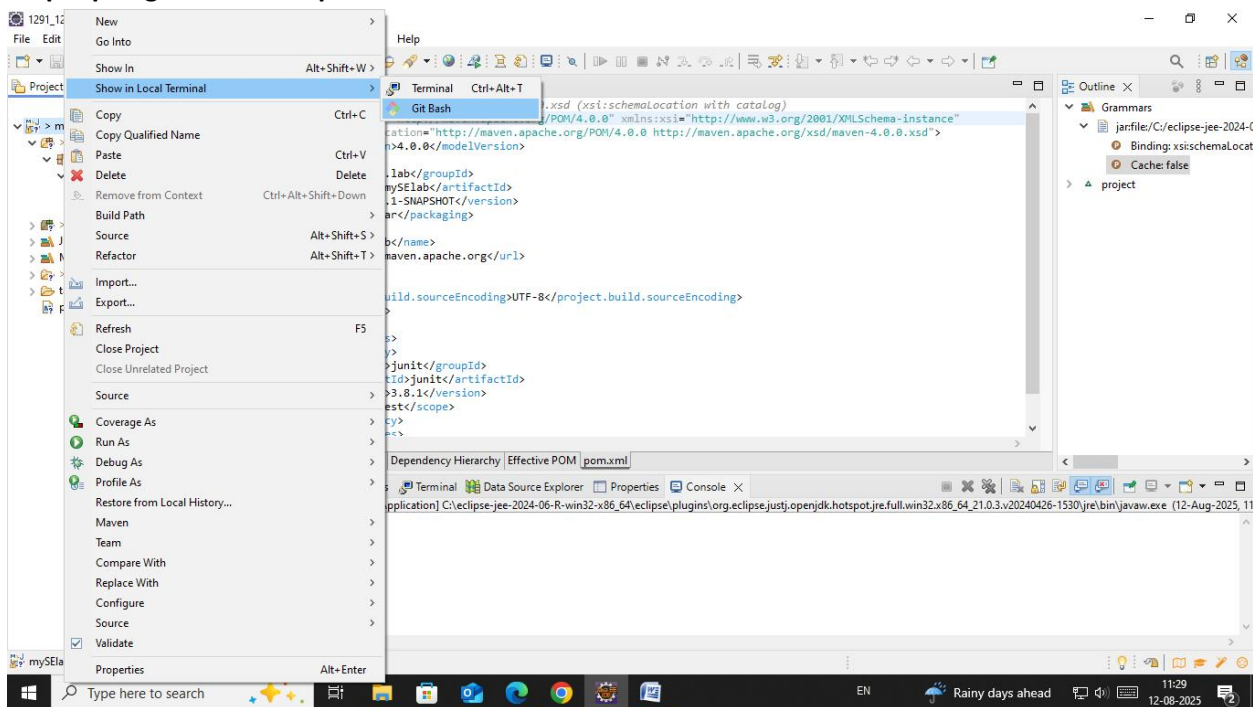
Rainy days ahead

11:29

12-08-2025



Step: open git bash in eclipse



Step: push maven java project into Git hub repository

1291_1285 - mySElab/pom.xml - Eclipse IDE

File Edit Source Navigate Search Project Run Design Window Help

Project Explorer

- mySElab [1291_1285 main]
 - src/main/java
 - com.lab.mySElab
 - App.java
 - App
 - main(String[]): void
 - src/test/java
 - JRE System Library [JavaSE-1.8]
 - Maven Dependencies
 - src
 - target
 - pom.xml

mySElab/pom.xml

http://maven.apache.org/xsd/maven-4.0.0.xsd (xsi:schemaLocation with catalog)
1<?project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
<
Overview | Dependencies | Dependency Hierarchy | Effective POM | pom.xml

Problems # Servers Terminal X Data Source Explorer Properties Console

MINGW64/c/Users/fs/Desktop/1291_1285/mySElab X

```
fs@DESKTOP-GGREQK6 MINGW64 ~/Desktop/1291_1285/mySElab (main)
$ git init
Initialized empty Git repository in C:/Users/fs/Desktop/1291_1285/mySElab/.git/

fs@DESKTOP-GGREQK6 MINGW64 ~/Desktop/1291_1285/mySElab (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .classpath
        .gitignore
        .project
        .settings/
        pom.xml
        src/

nothing added to commit but untracked files present (use "git add" to track)

fs@DESKTOP-GGREQK6 MINGW64 ~/Desktop/1291_1285/mySElab (master)
$ git add .
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the ne
xt time Git touches it

fs@DESKTOP-GGREQK6 MINGW64 ~/Desktop/1291_1285/mySElab (master)
$ git commit -m "javaproject"
[master (root-commit) 3e1b3e9] javaproject
 9 files changed, 156 insertions(+)
 create mode 100644 .classpath
 create mode 100644 .gitignore
 create mode 100644 .project
 create mode 100644 .settings/org.eclipse.core.resources.prefs
 create mode 100644 .settings/org.eclipse.jdt.core.prefs
 create mode 100644 .settings/org.eclipse.m2e.core.prefs
 create mode 100644 pom.xml
 create mode 100644 src/main/java/com/lab/mySElab/App.java
 create mode 100644 src/test/java/com/lab/mySElab/AppTest.java

fs@DESKTOP-GGREQK6 MINGW64 ~/Desktop/1291_1285/mySElab (master)
$ git remote add origin https://github.com/Rohith6936/SE_LAB_1291_12b5.git
Total 23 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'master' on GitHub by visiting:
remote:   https://github.com/Rohith6936/SE_LAB_1291_12b5/pull/new/master
remote:
To https://github.com/Rohith6936/SE_LAB_1291_12b5.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.

fs@DESKTOP-GGREQK6 MINGW64 ~/Desktop/1291_1285/mySElab (master)
$
```

Connected - Encoding: UTF-8

1291_1285 - mySElab/pom.xml - Eclipse IDE

File Edit Source Navigate Search Project Run Design Window Help

Project Explorer

- mySElab [1291_1285 main]
 - src/main/java
 - com.lab.mySElab
 - App.java
 - App
 - main(String[]): void
 - src/test/java
 - JRE System Library [JavaSE-1.8]
 - Maven Dependencies
 - src
 - target
 - pom.xml

mySElab/pom.xml

http://maven.apache.org/xsd/maven-4.0.0.xsd (xsi:schemaLocation with catalog)
1<?project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
<
Overview | Dependencies | Dependency Hierarchy | Effective POM | pom.xml

Problems # Servers Terminal X Data Source Explorer Properties Console

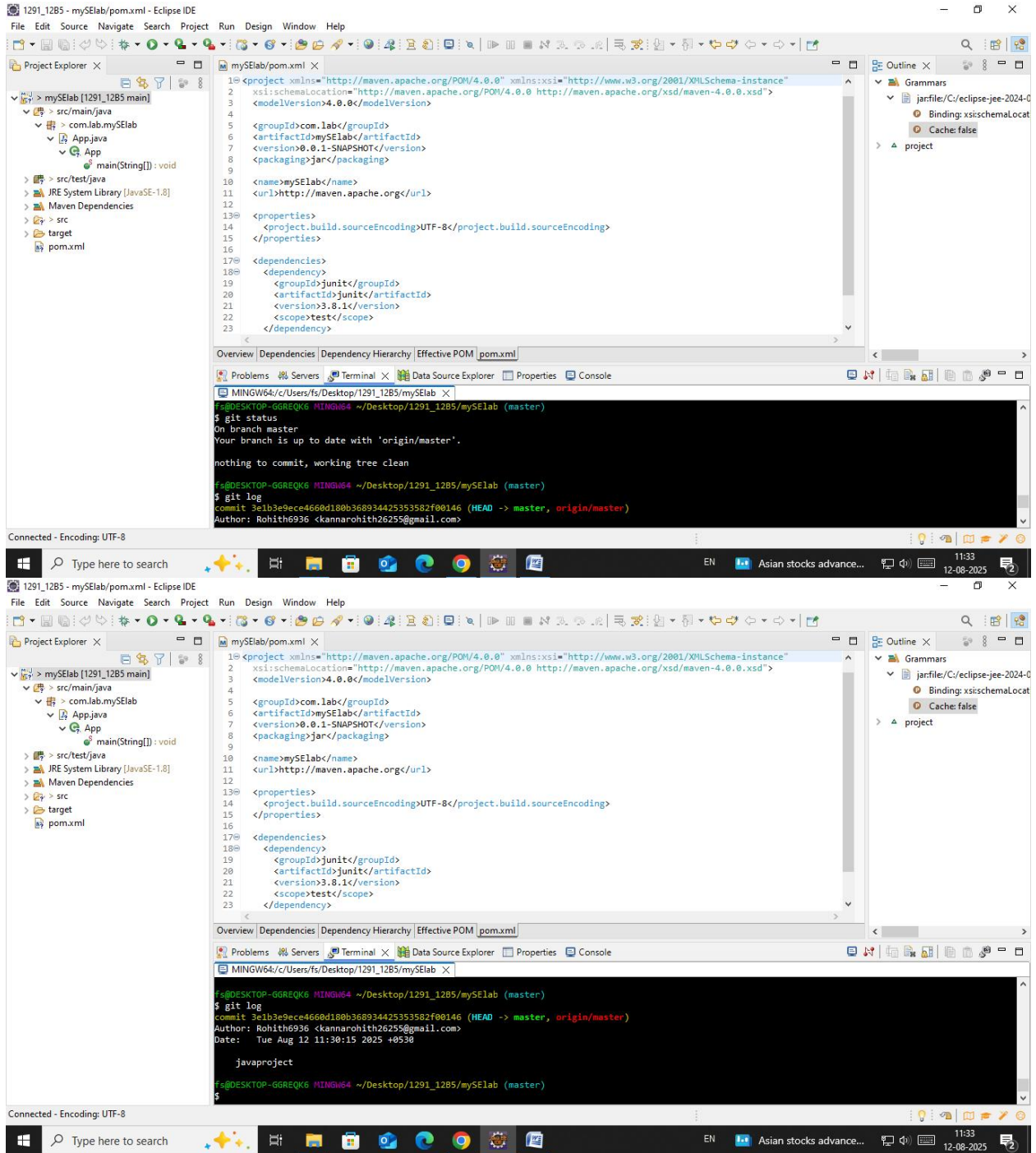
MINGW64/c/Users/fs/Desktop/1291_1285/mySElab X

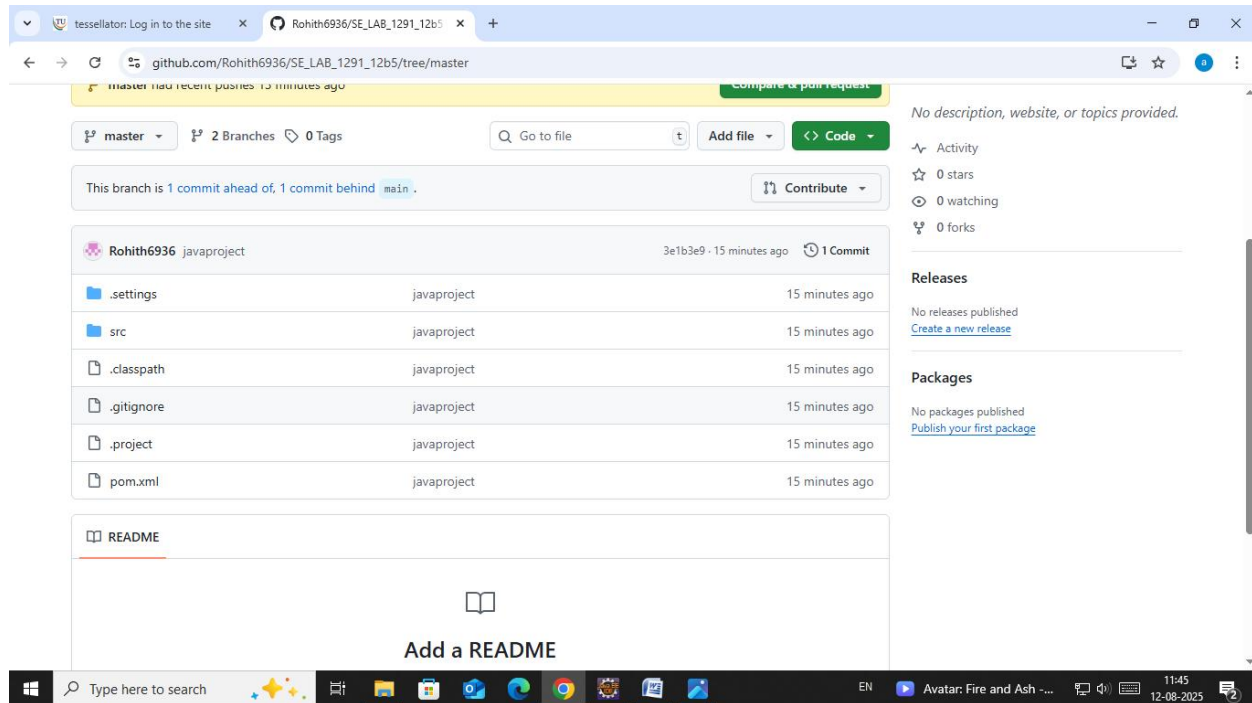
```
9 files changed, 156 insertions(+)
 create mode 100644 .classpath
 create mode 100644 .gitignore
 create mode 100644 .project
 create mode 100644 .settings/org.eclipse.core.resources.prefs
 create mode 100644 .settings/org.eclipse.jdt.core.prefs
 create mode 100644 .settings/org.eclipse.m2e.core.prefs
 create mode 100644 pom.xml
 create mode 100644 src/main/java/com/lab/mySElab/App.java
 create mode 100644 src/test/java/com/lab/mySElab/AppTest.java

fs@DESKTOP-GGREQK6 MINGW64 ~/Desktop/1291_1285/mySElab (master)
$ git remote add origin https://github.com/Rohith6936/SE_LAB_1291_12b5.git
Total 23 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'master' on GitHub by visiting:
remote:   https://github.com/Rohith6936/SE_LAB_1291_12b5/pull/new/master
remote:
To https://github.com/Rohith6936/SE_LAB_1291_12b5.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.

fs@DESKTOP-GGREQK6 MINGW64 ~/Desktop/1291_1285/mySElab (master)
$
```

Connected - Encoding: UTF-8



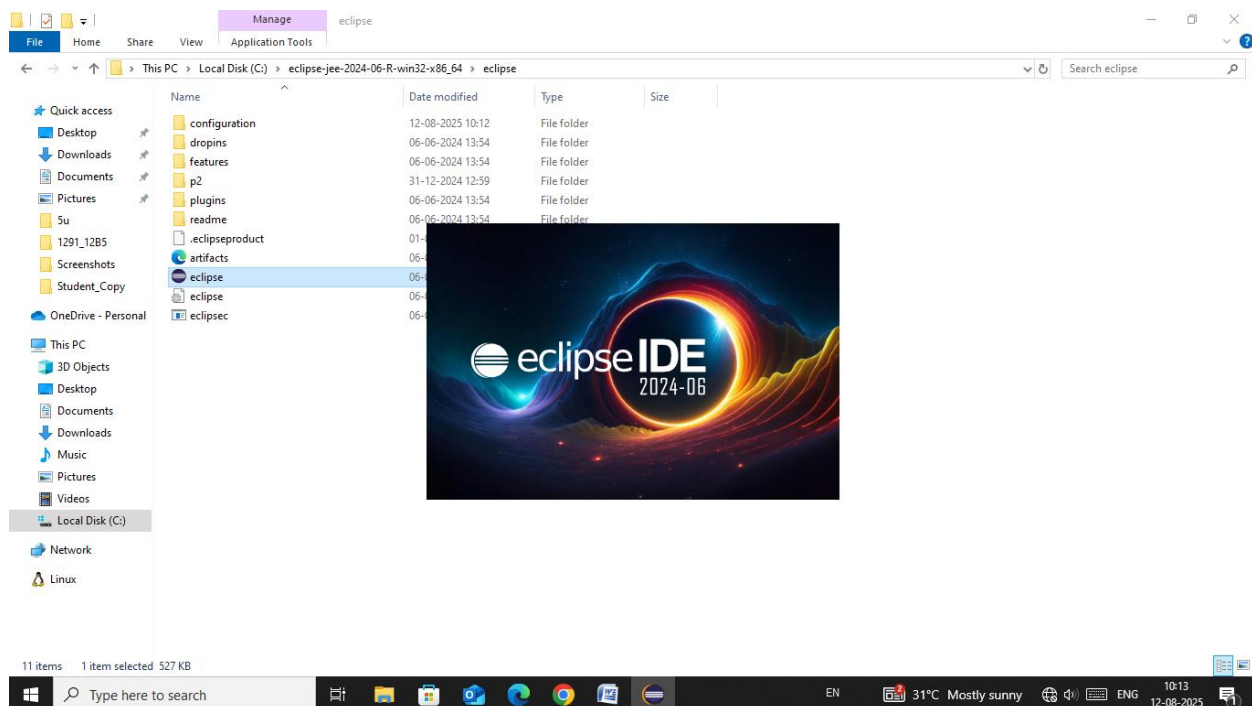


Creation of Maven web Java Project

Step 1: Open Eclipse

└─ 1.1 Launch Eclipse IDE.

└─ 1.2 Select or create a workspace.

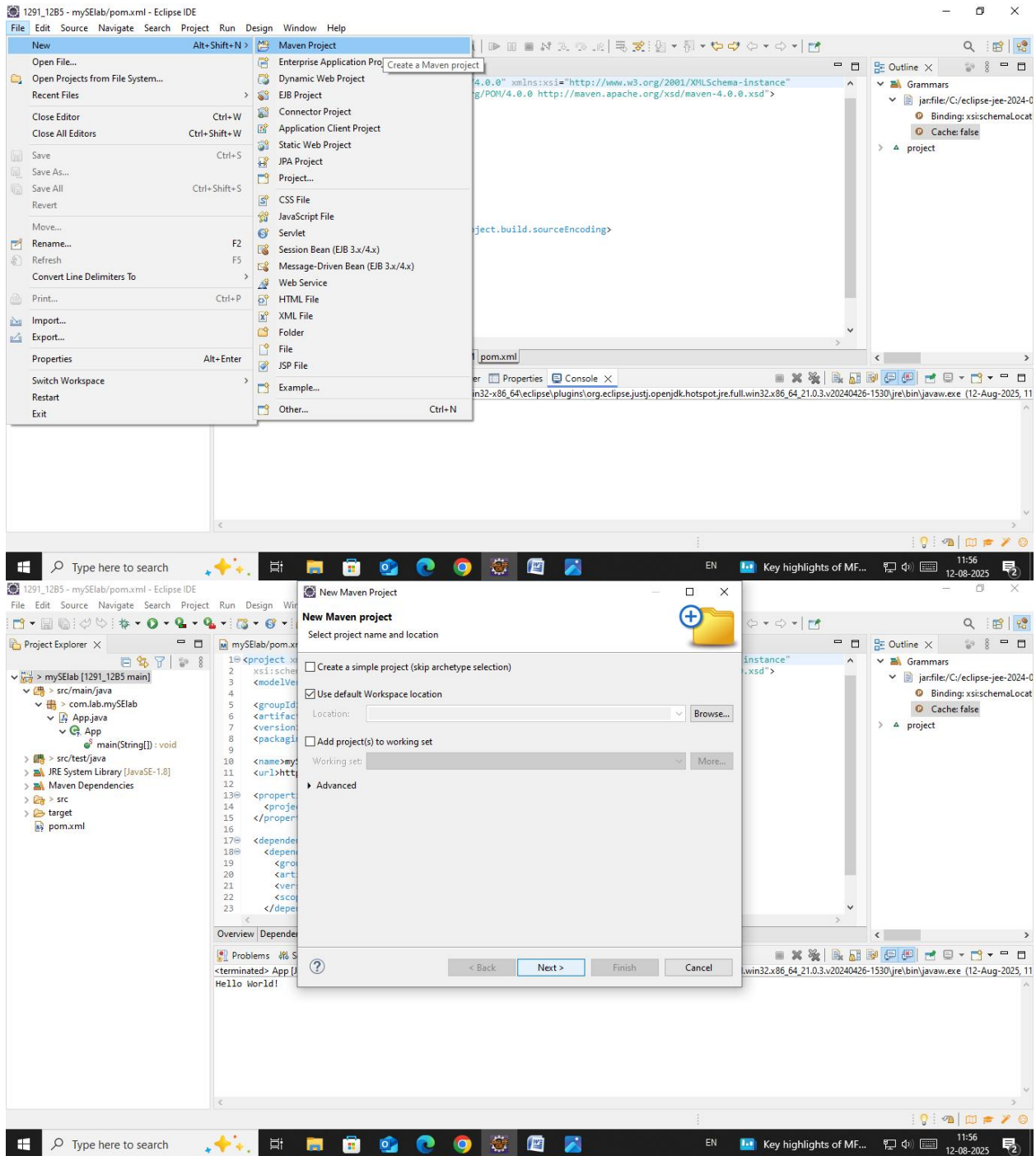


Step 2: Create a New Maven Project

└─ 2.1. File -> New -> Project...

└─ 2.1.1. Expand "Maven"

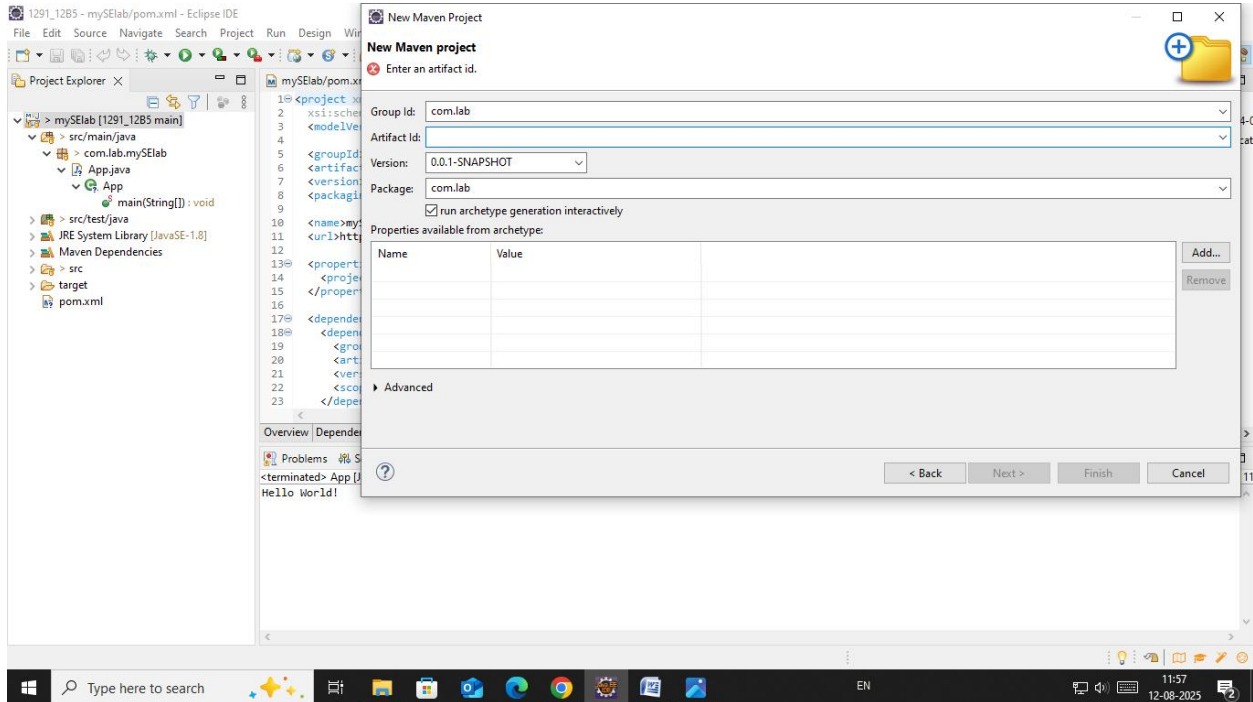
└─ 2.1.2. Select "Maven Project" and click "Next"



Step 3: Choose Maven Archetype

└─ 3.1. Select an archetype(e.g. "'org.apache.maven.archetypes' -> 'maven-archetype-webapp' 1.4 ")

└─ 3.2. Click "Next"

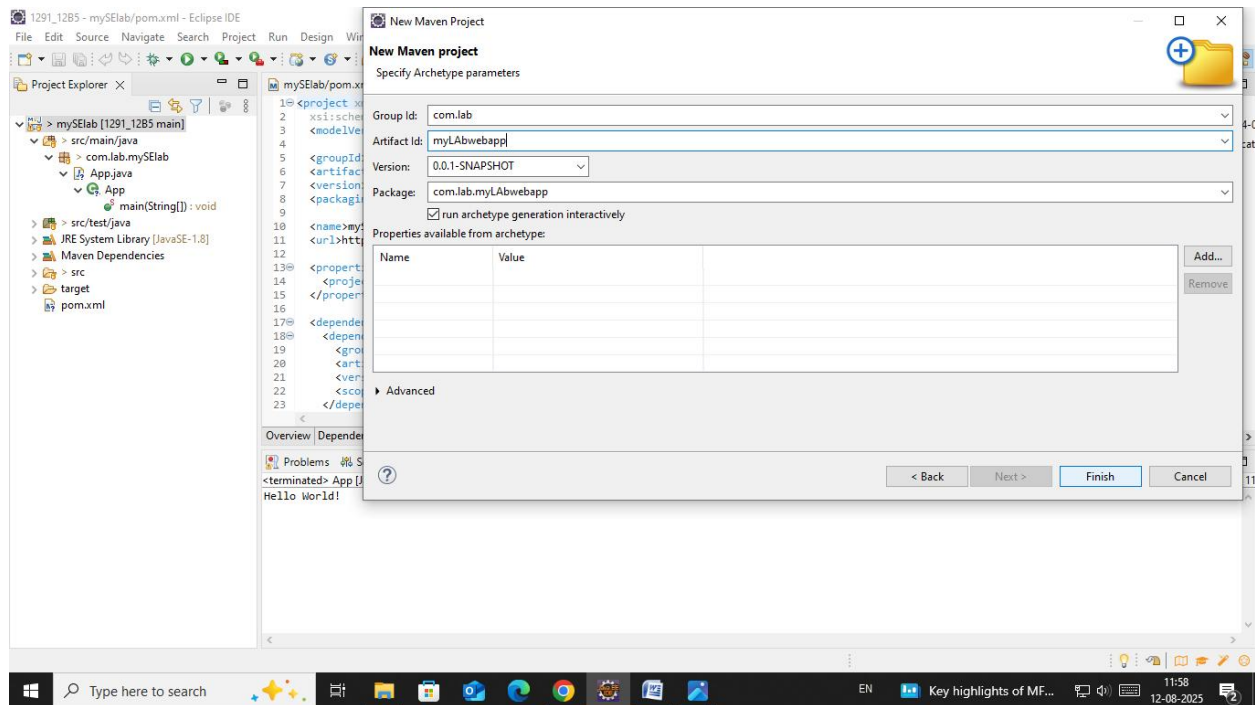


Step 4: Configure the Maven Project

└─ 4.1 Group Id: Enter a group ID (e.g., com.example).

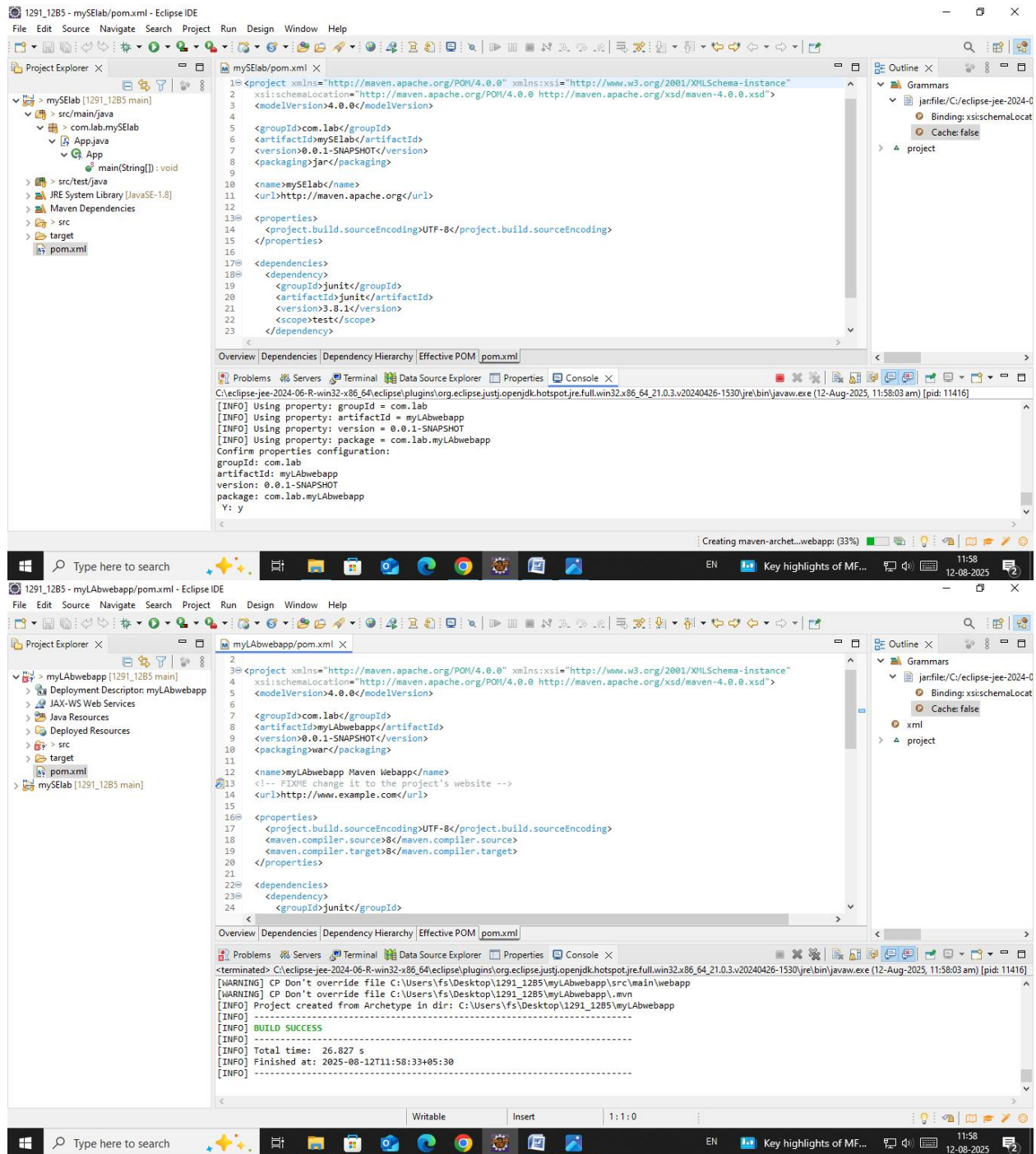
└─ 4.2 Artifact Id: Enter an artifact ID (e.g., my-web-app).

└─ 4.3 Click ****Finish**** to create the project.



Step 5: Add Maven Dependencies

└─ 5.1 Open the ****pom.xml**** file in the Maven project.



— 5.2 Add the necessary dependencies for your web project (e.g., Servlet, JSP):

Go to browser -> Open mvnrepository.com

Search for 'Java Servlet API' -> Select the latest version.

Copy the dependency code -> Paste it in MavenWeb's pom.xml under the target folder

The screenshot shows the Maven Repository search results for the query 'java.servlet-api'. The search bar at the top contains the text 'java.servlet-api'. Below the search bar, the results are sorted by 'Relevance'. The first result is '1. Java Servlet API' with 20,379 usages. It is described as 'Java Servlet is the foundation web specification in the Java Enterprise Platform. Developers can build web applications using the Servlet API to interact with the request/response workflow.' The last release was on Apr 20, 2018. A note indicates it has been 'Relocated' from 'javax.servlet' to 'jakarta.servlet-api'. The second result is '2. JavaServlet(TM) Specification' with 13,614 usages, also indicating a relocation from 'javax.servlet' to 'jakarta.servlet-api'. The third result is '3. SLF4J API Module' with 77,619 usages, indicating a relocation from 'org.slf4j' to 'slf4j-api'. On the left, there are lists for 'Repository' (Central, Sonatype, etc.) and 'Group' (com.github, io.github, etc.). On the right, there are sections for 'Indexed Repositories (3128)' and 'Popular Tags'.

The screenshot shows the Maven Repository artifact page for 'jakarta.servlet-api'. The breadcrumb trail is 'Home » jakarta.servlet » jakarta.servlet-api'. The main heading is 'Jakarta Servlet', with a description: 'API for Jakarta Servlet, enabling web applications to handle HTTP requests and responses through a standard interface for building dynamic web content.' The license is 'EPL 2.0 | GPL'. The categories are 'Java Specifications'. The tags are 'jakarta', 'standard', 'servlet', 'api', 'specs'. The homepage is 'https://projects.eclipse.org/projects/ee4j.servlet'. The ranking is '#87 in MvnRepository (See Top Artifacts)' and '#6 in Java Specifications'. It is used by 6,927 artifacts. Below this, there is a table showing the version history across different repositories. The table has columns for 'Version', 'Vulnerabilities', 'Repository', 'Usages', and 'Date'. The versions listed are 6.1.0, 6.1.x (6.1.0-M2, 6.1.0-M1), and 6.0.x (6.0.0, 5.0.0). The usages for 6.1.0 are 1,425, for 6.1.0-M2 is 82, for 6.1.0-M1 is 55, for 6.0.0 is 3,444, and for 5.0.0 is 1,773. The dates are Jun 11, 2024, Feb 27, 2024, Nov 23, 2023, Jun 01, 2022, and Sep 30, 2020. On the left, there is a section for 'Indexed Artifacts (61.2M)' with a bar chart showing growth over time. Below that is a list of 'Popular Categories'. On the right, there is an advertisement for KLM Royal Dutch Airlines and 'the Real'.

Example:

```
``xml
```

```
<dependency>
```

```
<groupId>javax.servlet</groupId>
```


<artifactId>javax.servlet-api</artifactId>

<version>4.0.1</version>

<scope>provided</scope>

</dependency>

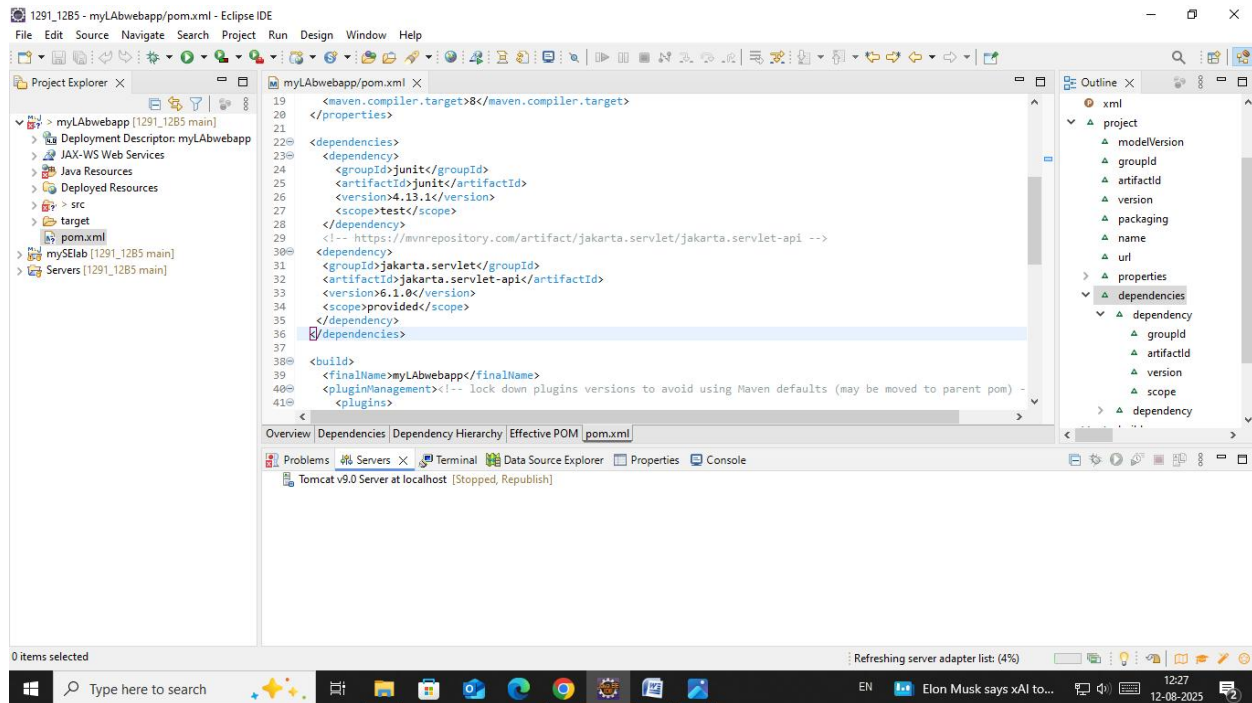
...

The screenshot shows a web browser window with the URL `mvnrepository.com/artifact/jakarta.servlet/jakarta.servlet-api/6.1.0`. The page displays the following information:

- Repositories:** Central | Talend Public
- Ranking:** #87 in MvnRepository (See Top Artifacts), #6 in Java Specifications
- Used By:** 6,927 artifacts
- Scope:** Provided
- Code Snippet:**

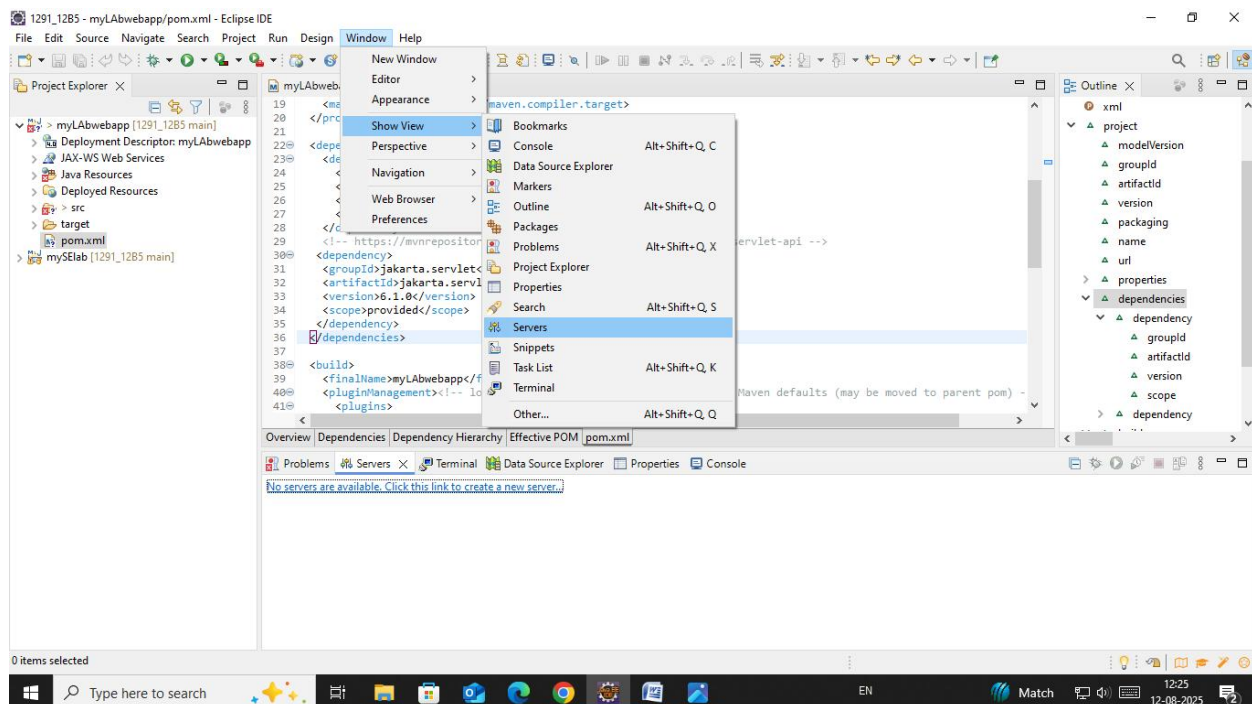
```
<!-- https://mvnrepository.com/artifact/jakarta.servlet/jakarta.servlet-api -->
<dependency>
  <groupId>jakarta.servlet</groupId>
  <artifactId>jakarta.servlet-api</artifactId>
  <version>6.1.0</version>
  <scope>provided</scope>
</dependency>
```
- Include backlinks:** ☒
- Compile Dependencies (0):** A table with columns: Category/License, Group / Artifact, Version, Updates.
- Popular Tags:** aar, android, apache, api, application, arm, assets, build, build-system, bundle, client, clojure, cloud, config, cran, data, database, eclipse, example, extension, framework, github, gradle, groovy, io, ios, javascript, kotlin, library, logging, maven, mobile, module, npm, osgi, plugin, resources, rlang, sdk, server, service, spring, sql, starter, testing, tools, ui, war, web, webapp.
- About:** Web site developed by @frodriguez. Powered by: Scala, Play, Spark, Pekko and Cassandra.

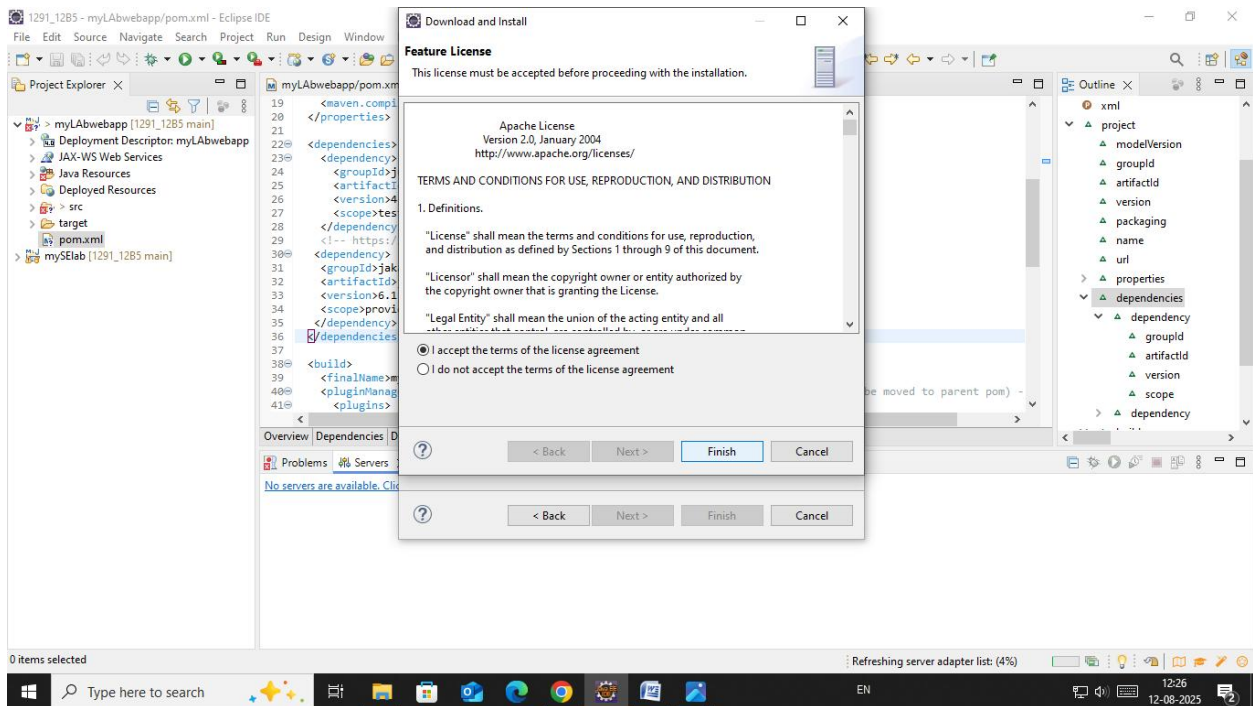
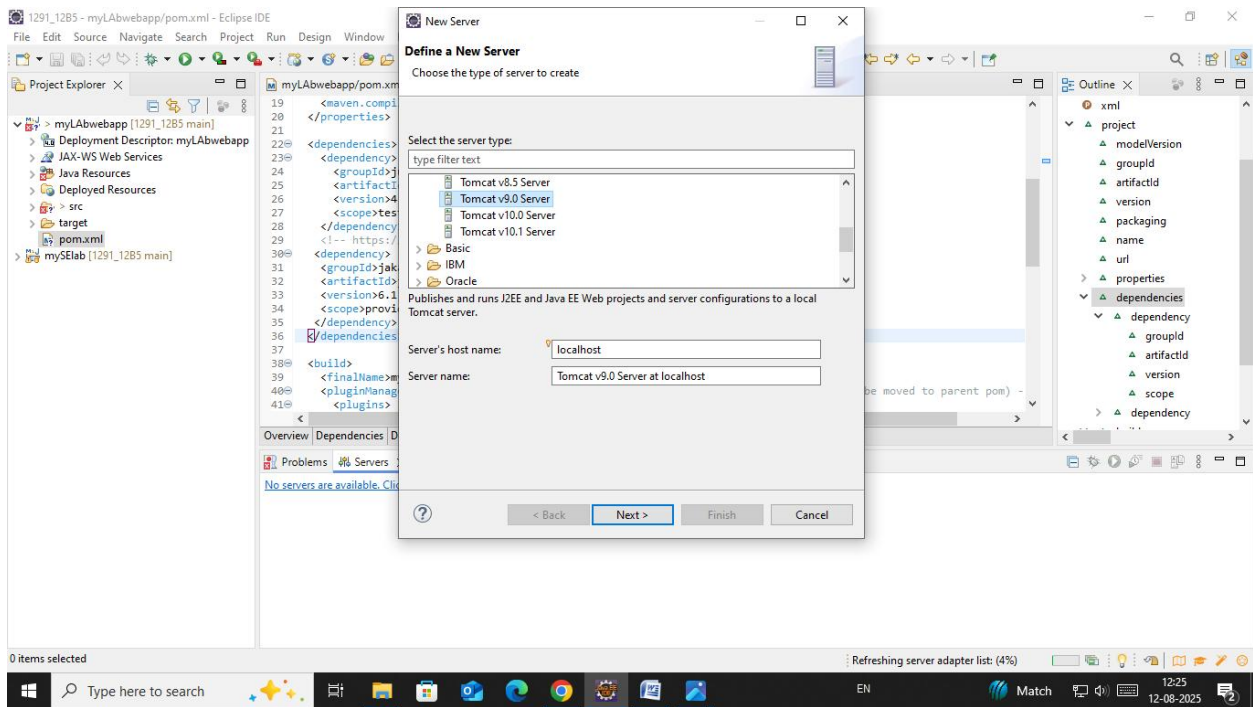
The Windows taskbar at the bottom shows the search bar, task view button, and several application icons. The system tray on the right shows the date and time as 12:21 on 12-08-2025.

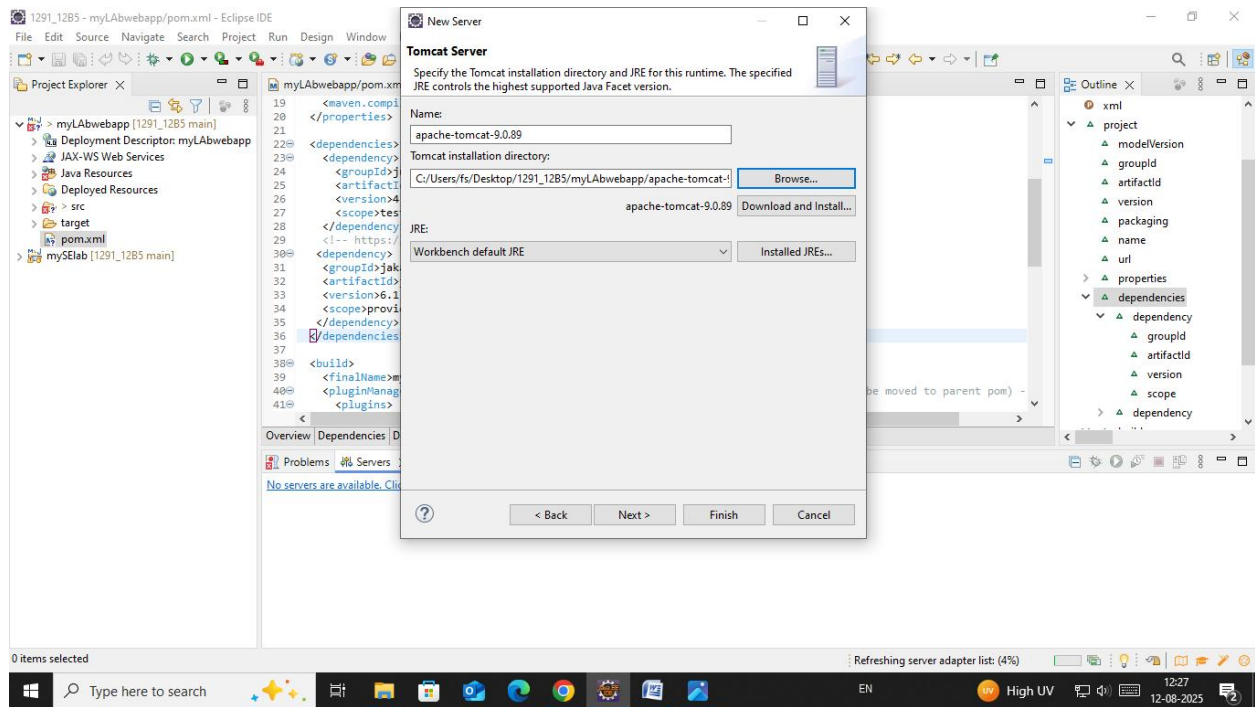


Step 6:- Configure server:

- Window -> Show View -> Servers
- Add server -> Select Tomcat v9.0 server -> Click Next
- Configure server options (e.g., ports, server location).

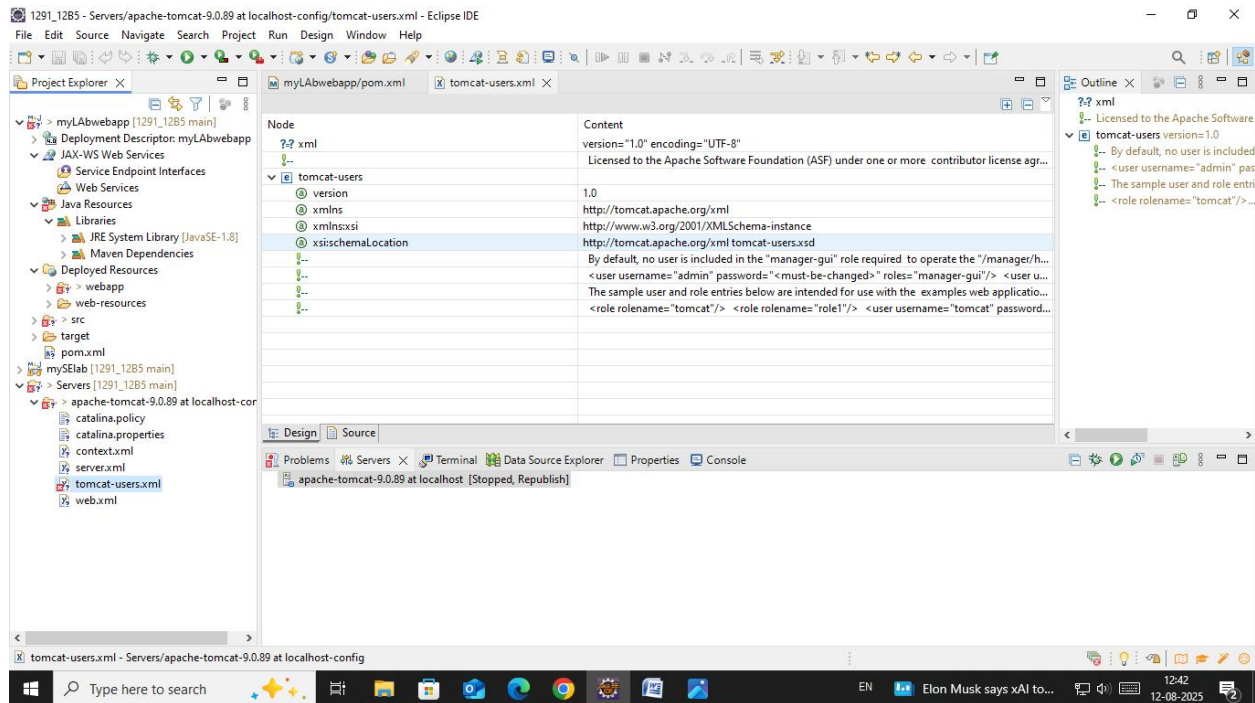






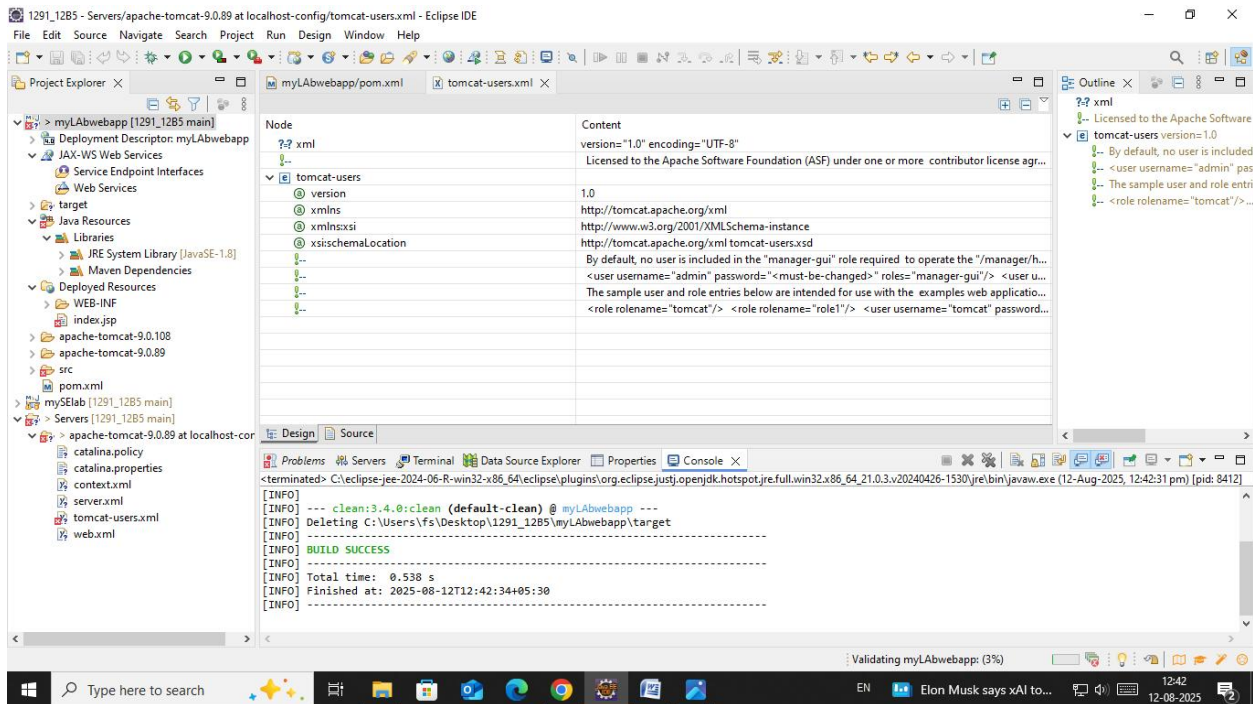
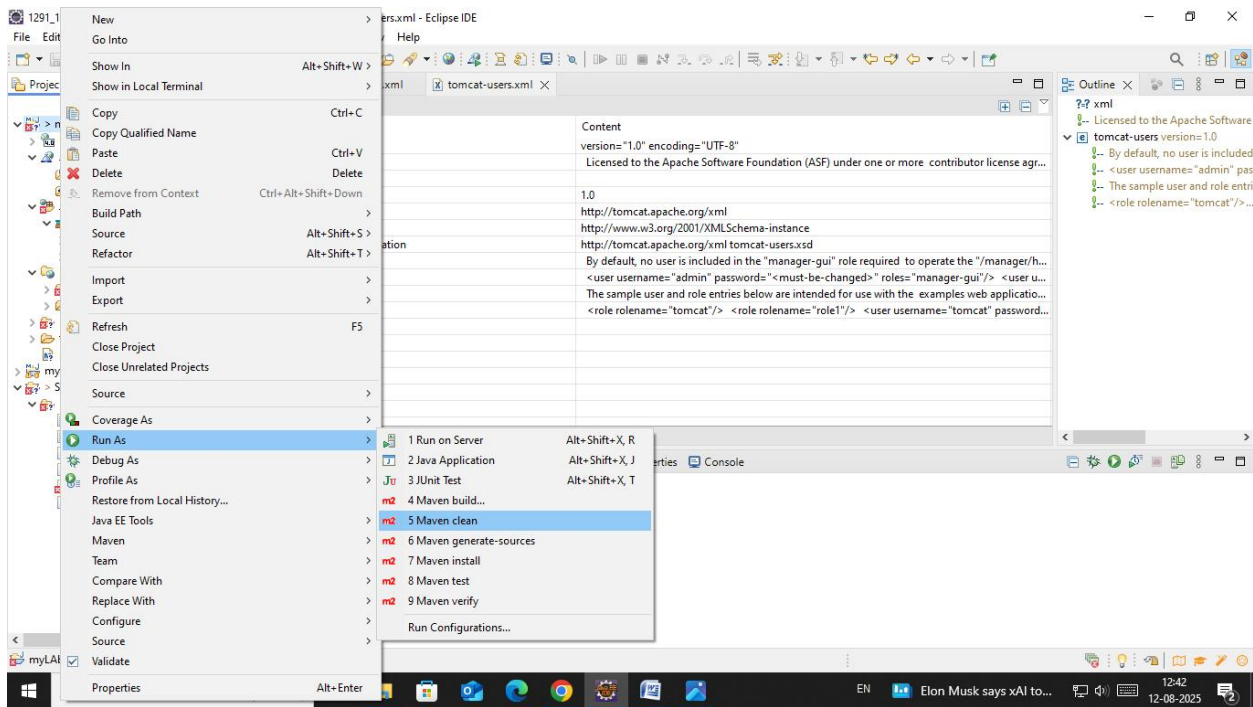
Step 7:- Modify 'tomcat-users.xml':

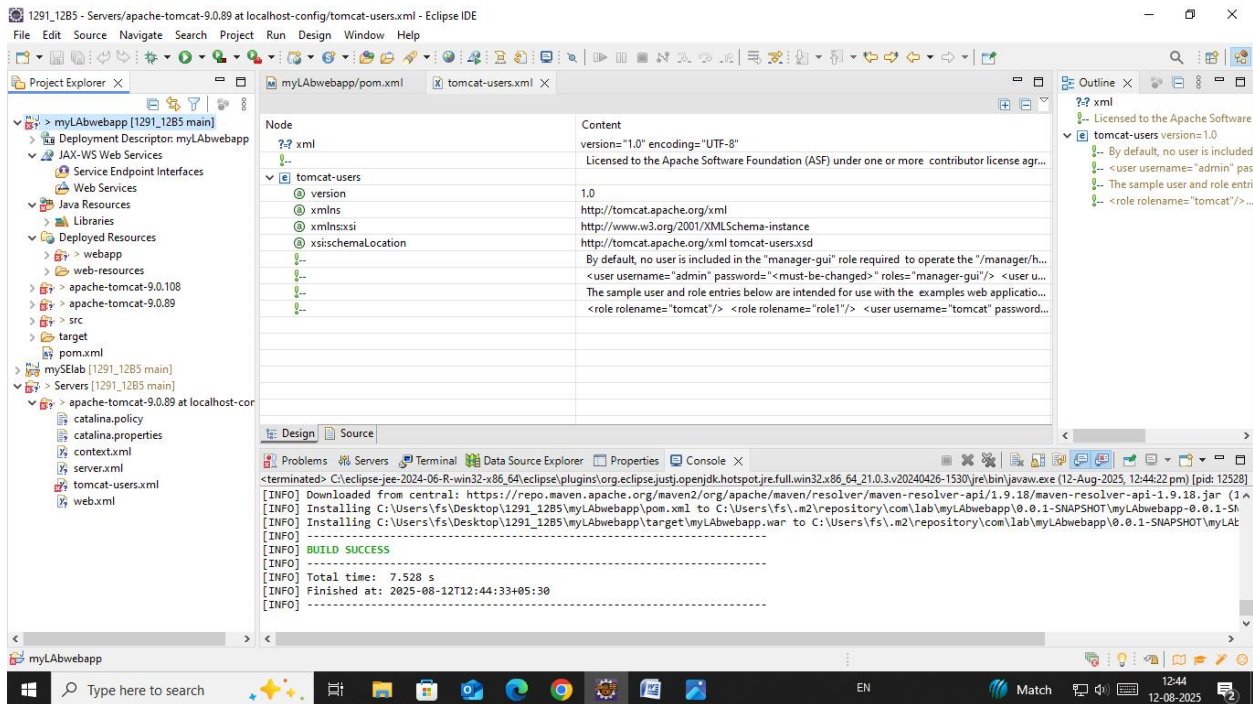
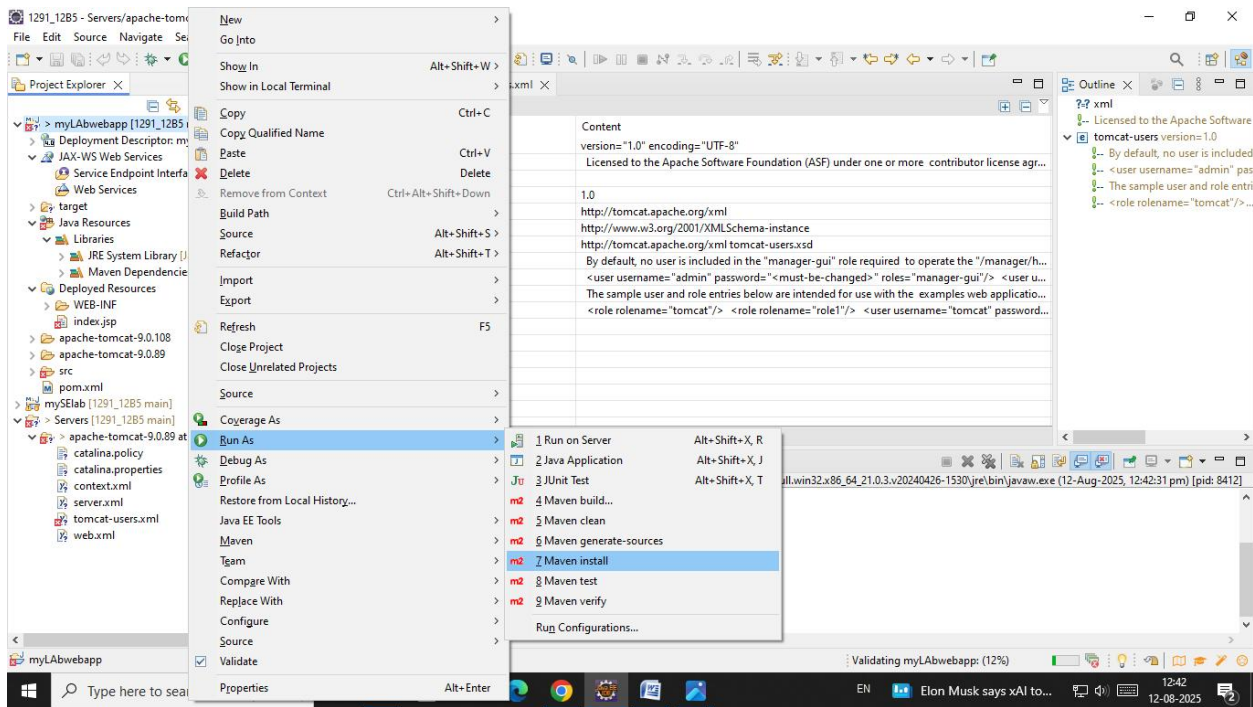
└─ Add role and user details under <tomcat-users> tag.

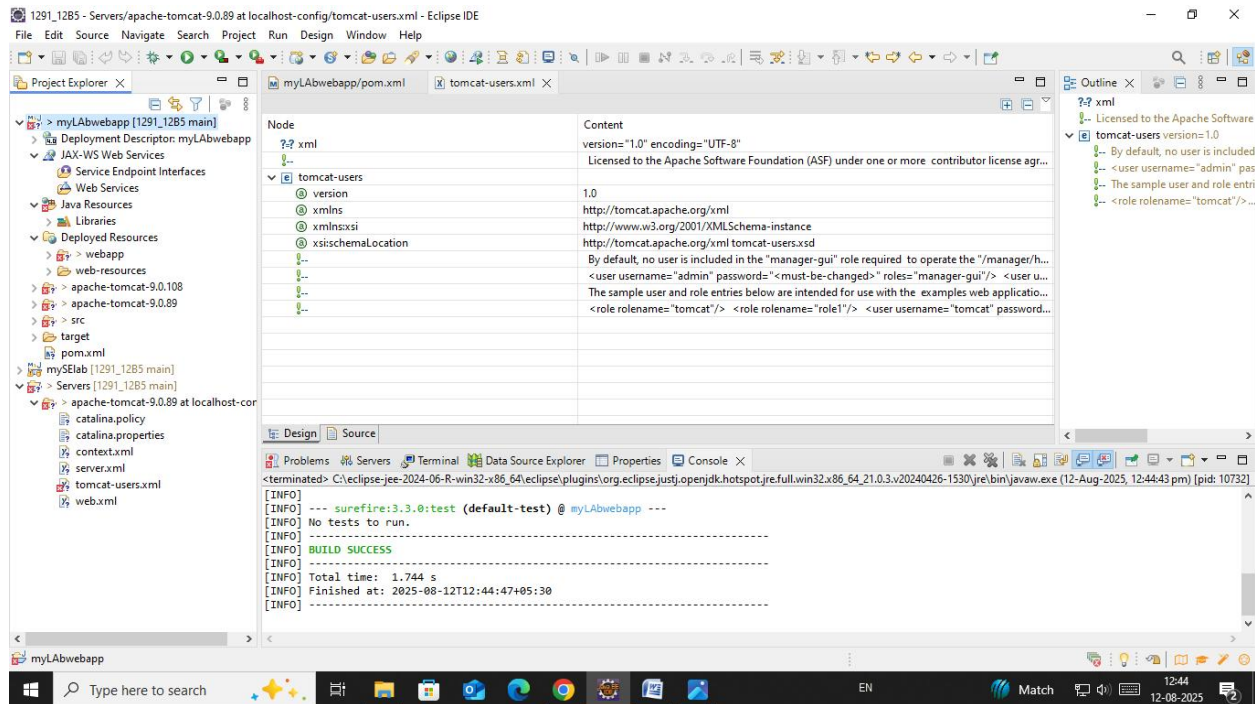
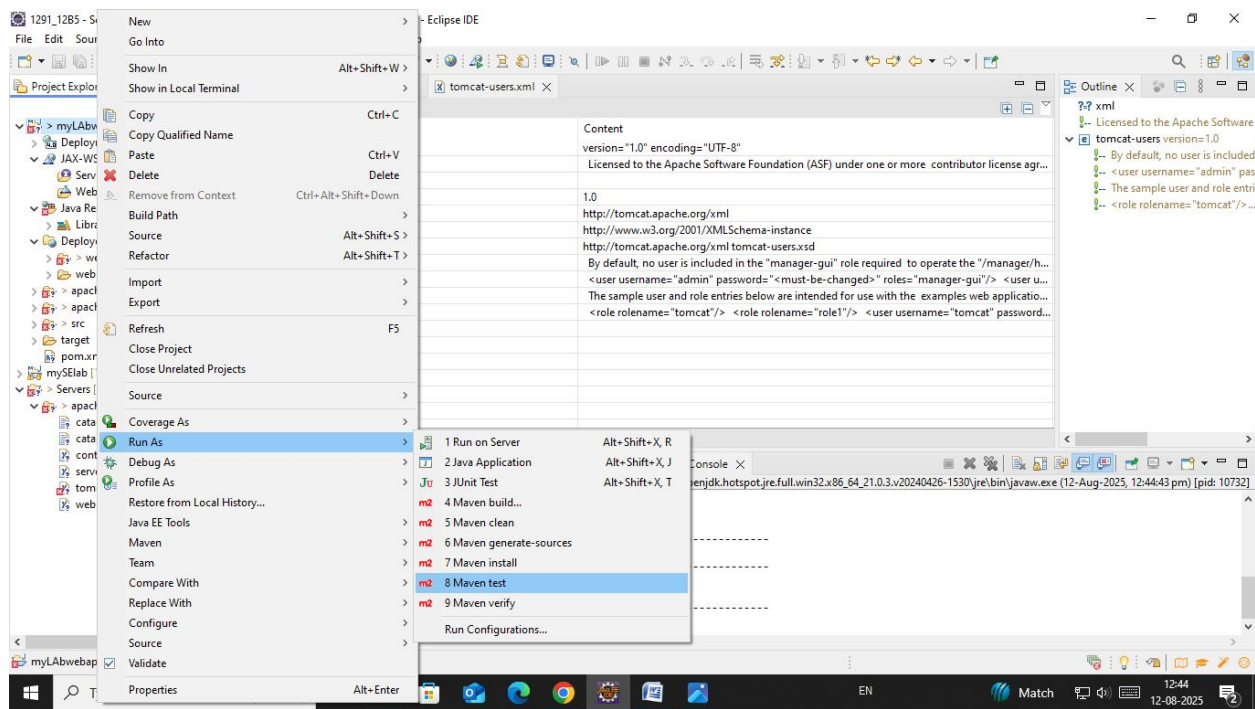


Step 8:. Build the project:

- └─ Right-click on index.jsp -> Run As -> Maven Clean
- └─ Right-click on index.jsp -> Run As -> Maven Install
- └─ Right-click on index.jsp -> Run As -> Maven Test
- └─ Right-click on index.jsp -> Run As -> Maven Build



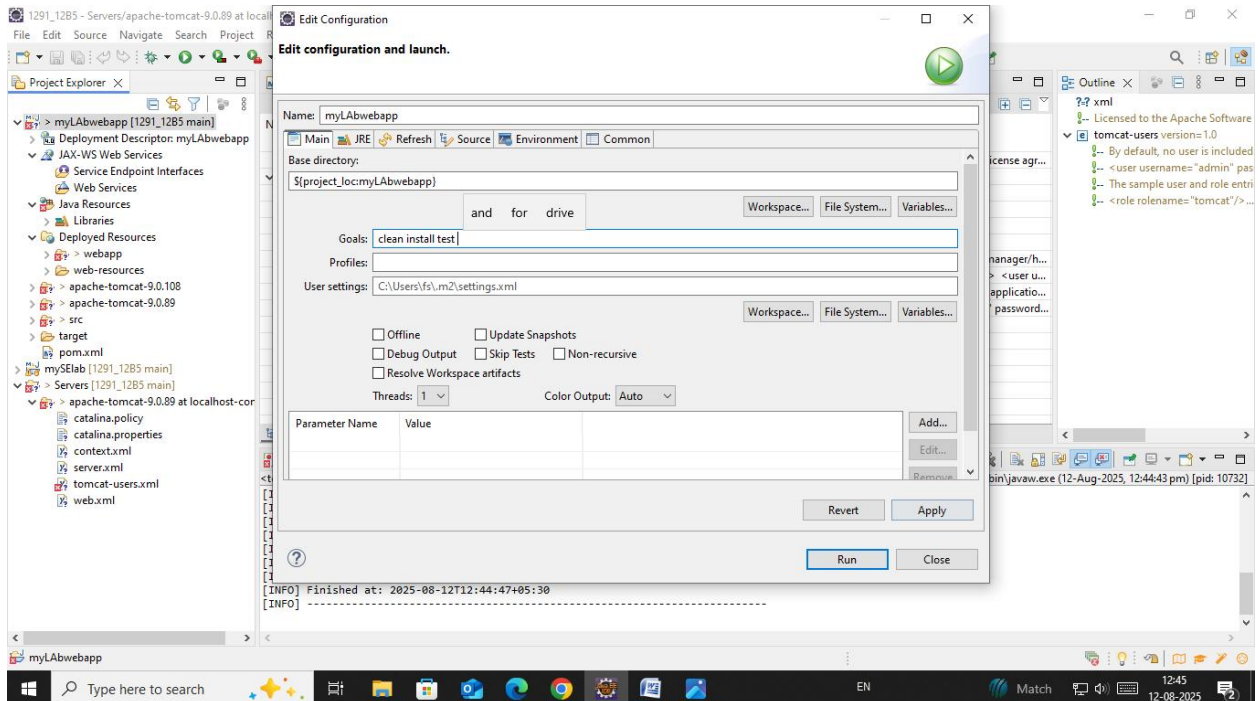
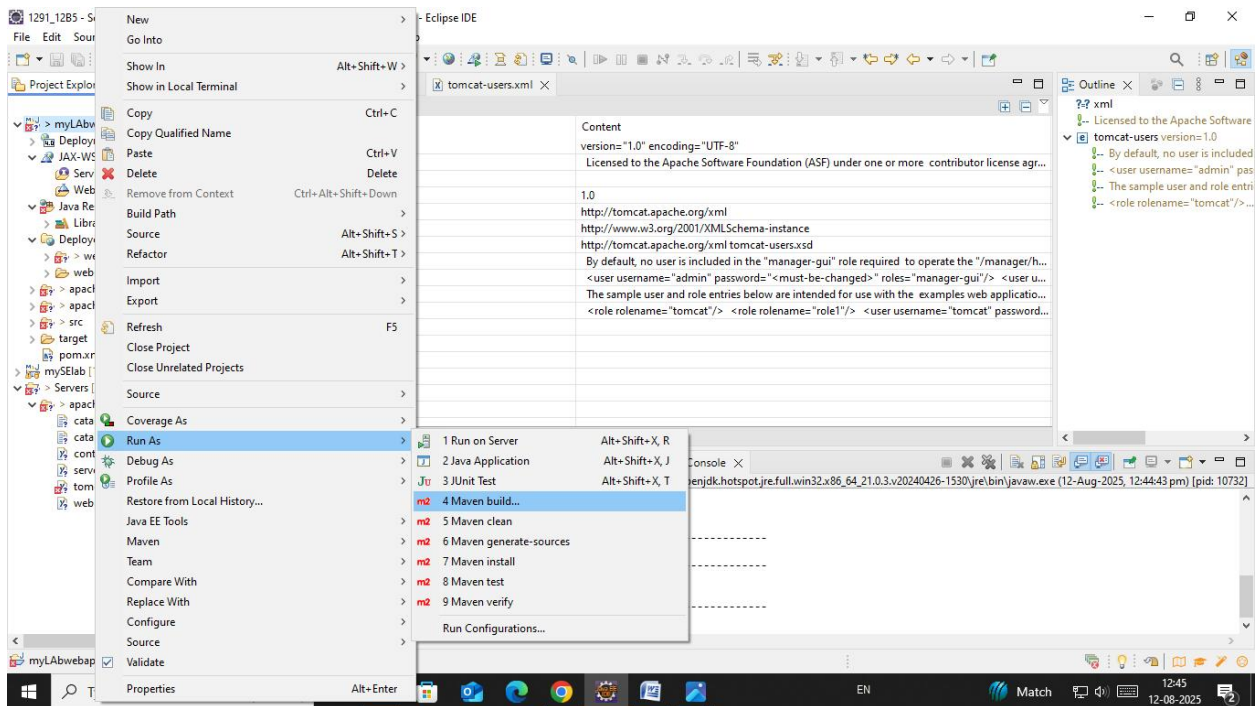


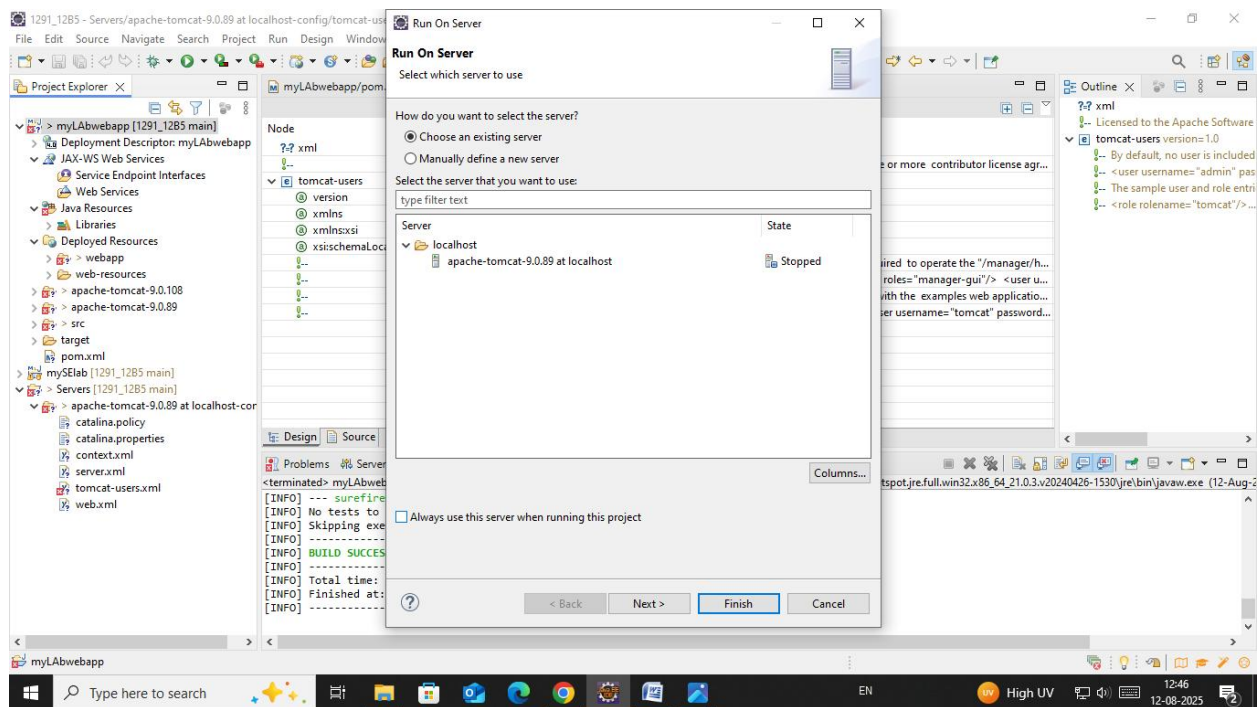


Step 9. In the Maven Build dialog:

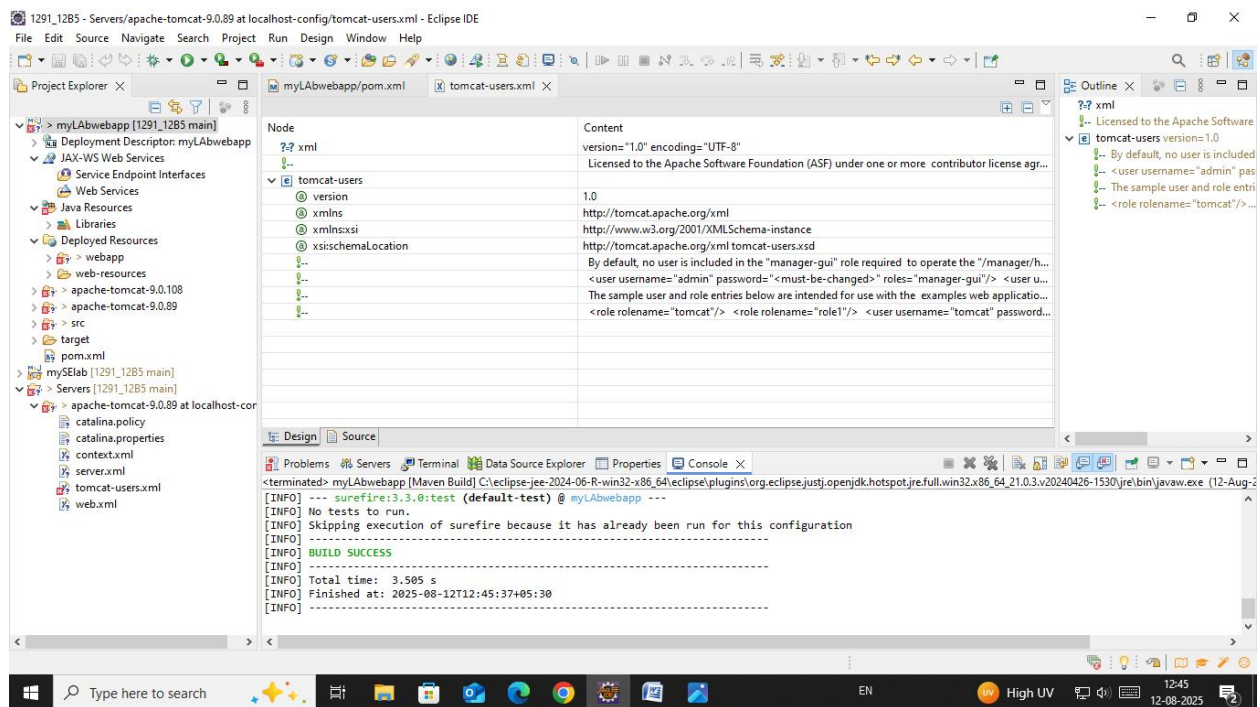
└ Enter Goals: clean install test

└ Click on Apply -> Click on Run





Step 10. Check console for BUILD SUCCESS message.



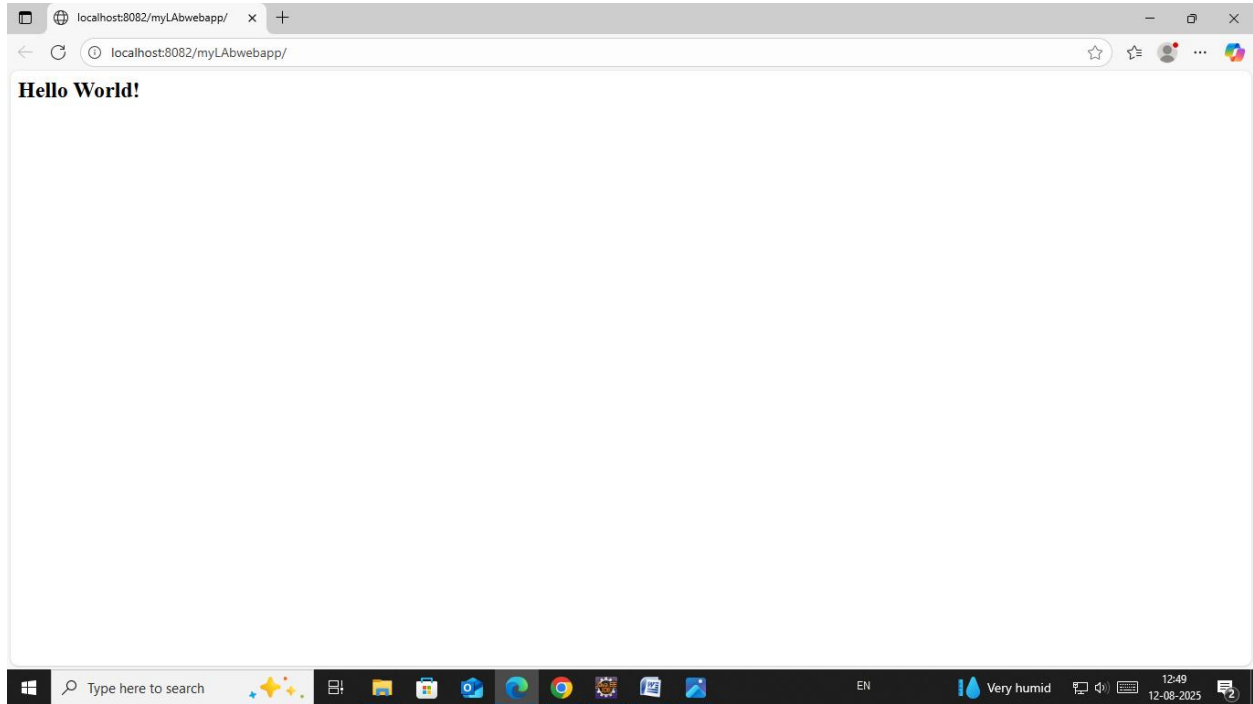
Step 11. Run the application:

└─ Right-click on index.jsp -> Run As -> Run on Server

└─ Select the Tomcat server -> Click on Finish

Step 12. Output: "Hello World" webpage displayed.

Note:-Now push yours Maven java project and Maven Web Project into your github



3. Configure Java Version via Compiler Plugin

In `pom.xml`:

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.11.0</version>
  <configuration>
    <source>17</source>
    <target>17</target>
  </configuration> </plugin>
```

Check:

- Run `mvn package`
- Inspect generated JAR: `jar tf target/myapp.jar`

4. JUnit Testing and Reports

Place test files in `src/test/java`. Example:

```
public class AppTest {  
    @Test  
    public void testSum() {  
        assertEquals(5, 2 + 3);  
    }  
}
```

Run:

`mvn test`

Check:

- `target/test-classes/` → *compiled test .class files*
- `target/surefire-reports/` → *.txt or .xml test results*
If test fails → corresponding log and failure trace shown

5. Handling Errors in pom.xml

- Typos in version numbers or missing repositories cause BUILD FAILURE
- Maven shows precise error in console
- *Fix the dependency tag → re-run* `mvn clean install`

6. Adding Resource Files

Put `config.properties` inside:

`src/main/resources/config.properties`

To read:

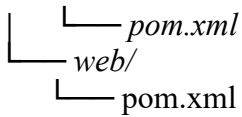
```
InputStream input =  
getClass().getClassLoader().getResourceAsStream("config.properties");
```

After build, check target/classes/ → file should exist there.

7. Multi-Module and Parent Projects

Structure:

```
parent/  
├── pom.xml (packaging: pom)  
└── core/
```



- Parent `pom.xml` defines `<modules>` and common dependencies
- Each submodule builds independently into its `target/` directory

8. Executable JARs

Add to `pom.xml`:

```

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-jar-plugin</artifactId>
      <configuration>
        <archive>
          <manifest>
            <mainClass>com.example.Main</mainClass>
          </manifest>
        </archive>
      </configuration>
    </plugin>
  </plugins>
</build>

```

Run:

```

mvn package
java -jar target/myapp.jar

```

9. Building a WAR File

Create a Maven web project with structure:

```

src/main/webapp/
└─ WEB-INF/web.xml

```

Add:

```

<packaging>war</packaging>

```

Command:

mvn package

Generates target/mywebapp.war → deploy on Tomcat server.

Conclusion

Mastering Maven empowers students to structure projects efficiently, manage complex dependencies, and automate testing and packaging. By practicing real-world scenarios—such as resolving build errors, handling resource files, applying patches, and deploying to servers—students gain valuable hands-on experience aligned with professional software development workflows. Maven not only streamlines builds but also enforces standardization and reusability across projects. Through Maven, students learn efficient project management, dependency control, multi-module integration, and reproducible builds. Understanding Maven's lifecycle, plugin system, and error handling prepares students for professional DevOps and CI/CD workflows. This lab equips students with hands-on experience in packaging, testing, resolving conflicts, and deploying real-world Java and Web applications.